VOLUME 3

Design Patterns & Clean Architecture

GoF patterns in real Spring code, DDD, hexagonal architecture, refactoring

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

13. Design Patterns in Real Java & Spring Code

13.1 How to talk about patterns
13.2 Creational patterns
13.3 Structural patterns
13.4 Behavioral patterns
13.5 Patterns inside Spring — a map
13.6 Plugins with ServiceLoader — Factory + Strategy across jars
13.7 Enterprise patterns every Spring developer uses
13.8 Anti-patterns
13.9 Interview questions and answers

14. Clean Code, DDD & Hexagonal Architecture

14.1 Clean code rules that still matter at senior level
14.2 Code smells and their refactorings
14.3 Layered vs hexagonal vs clean/onion
14.4 Package by layer vs package by feature
14.5 DDD strategic design
14.6 DDD tactical design
14.7 Aggregate sizing and consistency boundaries
14.8 Anti-corruption layer
14.9 Modular monolith vs microservices
14.10 Technical debt, code review, API design, ArchUnit
14.11 Interview questions and answers

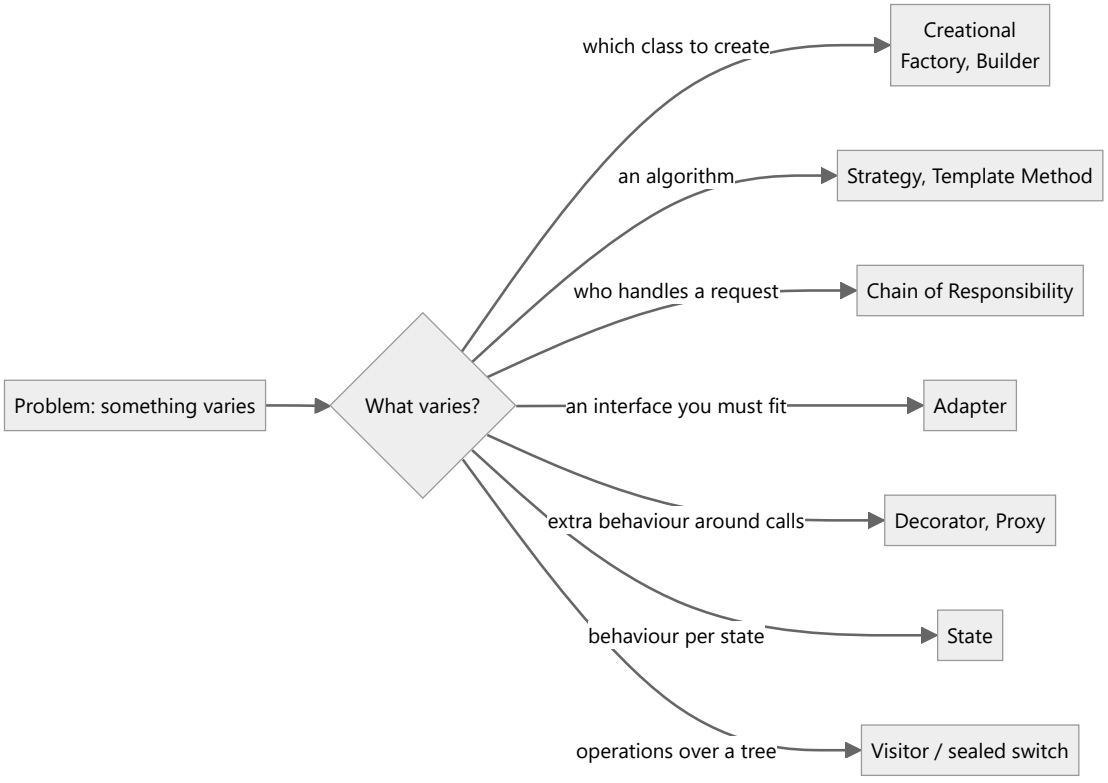
13. Design Patterns in Real Java & Spring Code

Pattern questions at senior level are never "draw the UML for Observer". The interviewer wants to know three things: can you recognise the pattern in code you already use every day (the JDK and Spring are full of them), do you know the *trade-off* each one buys, and can you point to a place in your own system where it earned its keep — or where it was overkill. This chapter goes through all 23 GoF patterns plus the enterprise patterns every Spring developer lives with, each with a small diagram, Java 21 code, where the JDK/Spring uses it, and a RefinePoint example where one honestly exists.

13.1 How to talk about patterns

A design pattern is a **named, reusable solution to a recurring design problem in a context**. The name is the valuable part: "put a decorator around the client" carries a whole design in five words.

Family	Question it answers	Patterns
Creational	How objects get created, and who decides the concrete class	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	How objects are composed into bigger structures	Adapter, Decorator, Proxy, Facade, Composite, Bridge, Flyweight
Behavioral	How objects distribute responsibility and talk to each other	Strategy, Template Method, Observer, Chain of Responsibility, Command, State, Visitor, Iterator, Mediator, Memento, Interpreter



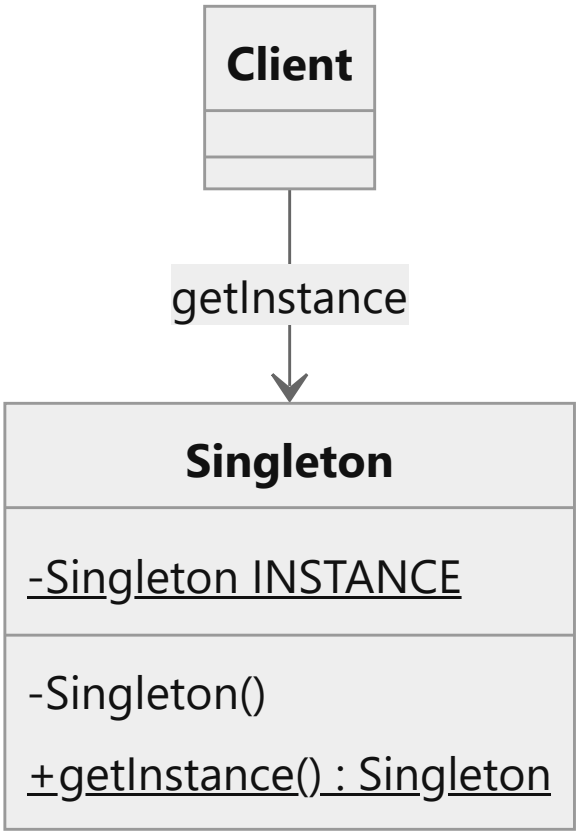
Three rules seniors follow:

1. **Patterns follow forces, not the other way round.** Introduce a strategy when the second algorithm arrives, not "in case".
2. **Modern Java dissolves some patterns.** Lambdas replace one-method Strategy/Command classes; records + sealed interfaces + pattern-matching `switch` replace classic Visitor; enums give you Singleton and simple State for free.
3. **Spring already implements many patterns for you** — singleton registry, factory, proxy, template, observer, chain. Recognising them is half of understanding Spring.

13.2 Creational patterns

Singleton — six implementations

Intent: exactly one instance per JVM (strictly: per class loader) with a global access point.



1. Eager initialisation — simplest, thread-safe because class initialisation is serialised by the JVM.

```
public final class EagerConfig {
    private static final EagerConfig INSTANCE = new EagerConfig();
    private EagerConfig() {}
    public static EagerConfig getInstance() { return INSTANCE; }
}
```

2. Static block — same as eager, but lets you handle exceptions during creation.

```
public final class KeyStoreHolder {
    private static final KeyStoreHolder INSTANCE;
    static {
```

```
        try { INSTANCE = new KeyStoreHolder(loadKeys()); }
        catch (IOException e) { throw new ExceptionInInitializerError(e); }
    }
    private KeyStoreHolder(KeyStore ks) { /* ... */ }
}
```

3. Lazy, synchronized method — correct but every call takes a lock.

```
public final class LazySync {
    private static LazySync instance;
    private LazySync() {}
    public static synchronized LazySync getInstance() {
        if (instance == null) instance = new LazySync();
        return instance;
    }
}
```

(The *unsynchronized* lazy version — same code without `synchronized` — is broken: two threads can both see `null` and create two instances.)

4. Double-checked locking (DCL) — lock only on first creation. `volatile` is mandatory.

```
public final class Dcl {
    private static volatile Dcl instance;
    private Dcl() {}
    public static Dcl getInstance() {
        Dcl local = instance;                // one volatile read on the fast path
        if (local == null) {
            synchronized (Dcl.class) {
                local = instance;
                if (local == null) instance = local = new Dcl();
            }
        }
        return local;
    }
}
```

Why `volatile`: `instance = new Dcl()` is three steps — allocate, run constructor, publish reference. Without a happens-before edge the JIT/CPU may publish the reference *before* the constructor finishes, so another thread sees a non-null, half-built object. `volatile` forbids that reordering (JSR-133, Java 5+).

5. Initialization-on-demand holder (Bill Pugh) — lazy *and* lock-free, using the JVM's class-init guarantee.

```
public final class Registry {
    private Registry() {}
    private static final class Holder { static final Registry INSTANCE = new Registry(); }
    public static Registry getInstance() { return Holder.INSTANCE; }   // Holder loads on first
call
}
```

6. Enum singleton — Effective Java's recommendation: serialization-safe and reflection-safe by language rule.

```
public enum IdGenerator {
    INSTANCE;
    private final AtomicLong next = new AtomicLong();
    public long nextId() { return next.incrementAndGet(); }
}
```

Implementation	Lazy	Thread-safe	Reflection-safe	Serialization-safe
Eager / static block	No	Yes	No (needs guard)	No (needs `readResolve`)
Synchronized method	Yes	Yes (slow)	No	No
Double-checked locking	Yes	Yes with `volatile`	No	No
Holder idiom	Yes	Yes	No	No
Enum	No (on first enum use)	Yes	**Yes**	**Yes**

Breaking a singleton and defending it

```
// 1. Reflection
Constructor<Registry> c = Registry.class.getDeclaredConstructor();
c.setAccessible(true);
Registry second = c.newInstance();                // a new instance

// Defence: fail in the constructor
private Registry() {
    if (Holder.INSTANCE != null) throw new IllegalStateException("Use getInstance()");
}

// 2. Serialization: readObject creates a new object
// Defence:
@Serial private Object readResolve() { return getInstance(); }

// 3. Cloning: only if the class (or a parent) implements Cloneable
@Override protected Object clone() throws CloneNotSupportedException { throw new
CloneNotSupportedException(); }
```

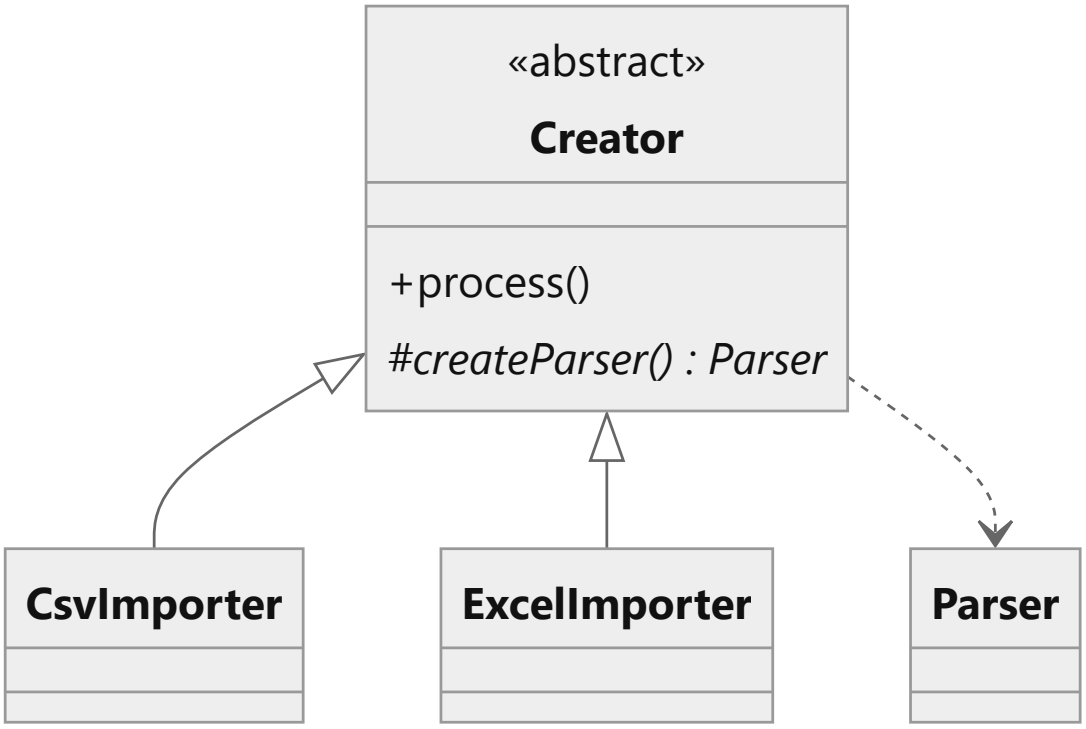
Other ways: **multiple class loaders** (each loads its own copy of the class — common in app servers and plugin systems) and **distributed deployment** (one per JVM, so five pods = five "singletons"). Enum is immune to reflection (`Constructor.newInstance` throws `IllegalArgumentException: Cannot reflectively create enum objects`) and serialization (enums serialize by name and resolve to the existing constant).

> **Trap:** Spring's "singleton" is not GoF Singleton. A Spring singleton is **one instance per bean definition per container** — you can have two beans of the same class, and two application contexts give two instances. Nothing stops `new MyService()` .

> **RefinePoint:** we have almost no hand-written singletons. Everything shared is a Spring singleton bean (`OutboxRelay` , `AuditTrail` , `LlmClient`), which keeps it injectable and mockable. The one static registry is `Plugins` in `ryfn-plugin-api` : a final class with a private constructor and a `volatile` class-loader field, because plugin discovery must work even outside Spring.

Factory Method

Intent: define a method for creating an object, and let the implementation (subclass or static factory) decide the concrete class.



Two flavours in practice:

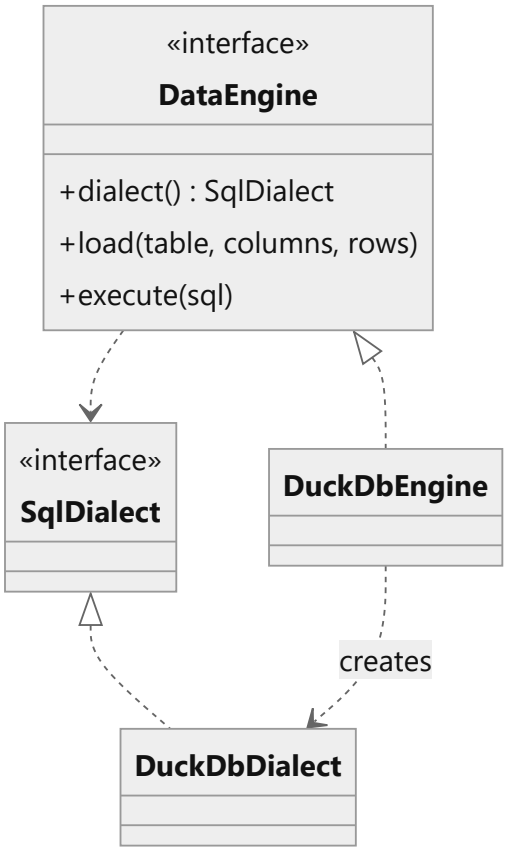
- **GoF factory method**: an abstract `createX()` overridden by subclasses.
- **Static factory method** (Effective Java item 1): `List.of`, `Optional.of`, `Integer.valueOf`, `Calendar.getInstance`, `LocalDate.of`, `EnumSet.of`. Advantages over constructors: a name, can return cached instances, can return any subtype (EnumSet returns `RegularEnumSet` or `JumboEnumSet` by size), can hide the class.

```
public sealed interface Parser permits CsvParser, ExcelParser, JsonParser {
    static Parser forFile(String fileName) {                // static factory method
        String ext = fileName.substring(fileName.lastIndexOf('.') + 1).toLowerCase();
        return switch (ext) {
            case "csv", "txt" -> new CsvParser();
            case "xlsx", "xls" -> new ExcelParser();
            case "json" -> new JsonParser();
            default -> throw new IllegalArgumentException("Unsupported file: " + fileName);
        };
    }
    List<Map<String, String>> parse(InputStream in);
}
```

Spring: `BeanFactory.getBean()` is the factory of factories; `FactoryBean<T>` is literally a factory method plugged into the container (`LocalContainerEntityManagerFactoryBean`, `SqlSessionFactoryBean`); `@Bean` methods are factory methods.

Abstract Factory

Intent: create **families** of related objects without naming concrete classes, so the family stays consistent.



JDK: `DocumentBuilderFactory`, `TransformerFactory`, JDBC `Connection` (creates matching `Statement`, `PreparedStatement`, `Blob` for one vendor). Swing look-and-feel is the textbook example.

> **RefinePoint:** `DataEngine` in `ryfn-engine` is our abstract-factory-shaped seam. A `DataEngine.Factory` opens an engine per run, and the engine hands out its own `SqlDialect`. `DuckDbEngine` returns the DuckDB dialect, so the SQL that `SqlCompiler` generates always matches the engine running it. A Spark engine would implement the same interface and return the Spark dialect — the family (engine + dialect + registered functions) can never be mixed.

Builder

Intent: construct a complex object step by step; separate construction from representation. Solves the **telescoping constructor** problem and gives readable, validated, immutable objects.

```
public final class ExportRequest {
    private final UUID runId; private final String format; private final boolean includeMatched;
private final ZoneId zone;
    private ExportRequest(Builder b) {
        this.runId = Objects.requireNonNull(b.runId, "runId");
        this.format = b.format; this.includeMatched = b.includeMatched; this.zone = b.zone;
    }
    public static Builder builder(UUID runId) { return new Builder(runId); }
    public static final class Builder {
        private final UUID runId; private String format = "xlsx"; private boolean includeMatched;
```

```
    private ZoneId zone = ZoneOffset.UTC;
        private Builder(UUID runId) { this.runId = runId; }
        public Builder format(String f) { this.format = f; return this; }
        public Builder includeMatched(boolean v) { this.includeMatched = v; return this; }
        public Builder zone(ZoneId z) { this.zone = z; return this; }
        public ExportRequest build() { return new ExportRequest(this); }   // validate here
    }
}
```

JDK/Spring: `HttpRequest.newBuilder()`, `HttpClient.newBuilder()`, `Stream.builder()`, `Locale.Builder`, `StringBuilder` (builder-ish), `UriComponentsBuilder`, `RestTemplateBuilder`, `WebClient.builder()`, `ResponseEntity.ok().header(...).body(...)`, `MockMvcRequestBuilders`, Spring Security's `HttpSecurity` DSL. Lombok `@Builder` generates it.

> **RefinePoint:** `AiConfig` builds the `RestTemplate` for the LLM provider with `RestTemplateBuilder` — `.connectTimeout(10s).readTimeout(props.getTimeoutSeconds())` — because a local GGUF model can take far longer than the platform's default timeouts. The builder is Boot's injected, pre-customised one (it already carries the message converters and observation setup), which is why we build from it instead of `new RestTemplate()`.

Records and builders: records have canonical constructors only; for 3–4 fields a record is enough. For many optional fields add a static nested builder or "wither" methods: `request.withFormat("csv")`.

Prototype

Intent: create new objects by **copying** an existing instance instead of building from scratch — useful when construction is expensive or configured at runtime.

Java's built-in mechanism is `Cloneable` / `Object.clone()`, which is broken by design (shallow, bypasses constructors, checked exception). Prefer copy constructors or copy factories:

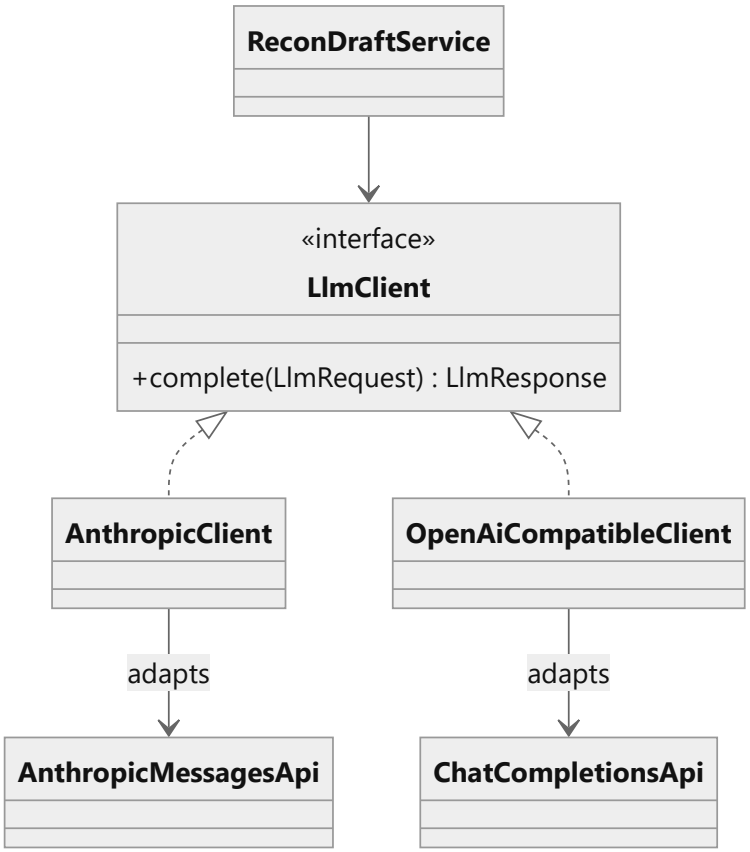
```
public record RuleTemplate(String name, List<String> keys, BigDecimal tolerance) {
    public RuleTemplate { keys = List.copyOf(keys); }
    public RuleTemplate withTolerance(BigDecimal t) { return new RuleTemplate(name, keys, t); }
// prototype-style copy
}
```

Spring's `prototype` scope is related only by name: the container creates a *new* instance per lookup by running the bean definition again — it does not clone an existing object.

13.3 Structural patterns

Adapter

Intent: convert the interface of a class into the interface clients expect.



JDK: `InputStreamReader` (bytes → chars), `Arrays.asList` (array → `List`), `Collections.enumeration`. Spring MVC's `HandlerAdapter` is the canonical example: `DispatcherServlet` only knows `HandlerAdapter.handle()`, and `RequestMappingHandlerAdapter`, `HttpRequestHandlerAdapter`, `SimpleControllerHandlerAdapter` adapt three different handler styles to it. Also `MessageListenerAdapter` (JMS) and Spring Security's `AuthenticationProvider` implementations.

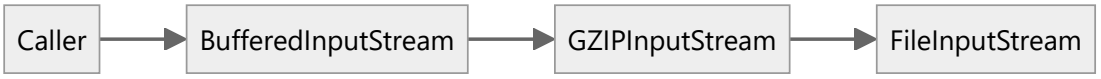
> **RefinePoint:** `LlmClient` is the target interface (records `LlmRequest`, `LlmResponse`, `Message`, `Attachment`). `AnthropicClient` adapts it to the Anthropic Messages API (system prompt as a separate field, content blocks, PDF attachments sent natively — `readsPdf()` returns true), and `OpenAiCompatibleClient` adapts it to the chat-completions format of our local model server. `ReconDraftService`, rule building and break explanation never see a provider type.

Object adapter (composition, holds the adaptee) is preferred over class adapter (inherits the adaptee) in Java because of single inheritance.

Decorator

Intent: attach responsibilities to an object dynamically by wrapping it in an object with the **same interface**.

```
InputStream in = new BufferedInputStream(new GZIPInputStream(new
FileInputStream("ledger.csv.gz")));
```



JDK: the whole `java.io` stream hierarchy, `Collections.unmodifiableList`, `Collections.synchronizedMap`, `Collections.checkedList`. Spring: `HttpServletRequestWrapper` / `ContentCachingRequestWrapper`, `TransactionAwareDataSourceProxy`, `LazyConnectionDataSourceProxy`, `TaskDecorator` (wraps `Runnable`s to propagate MDC/security context), `BeanPostProcessor`s that return wrappers.

```
final class TimedLlmClient implements LlmClient {           // decorator
    private final LlmClient delegate; private final MeterRegistry meters;
    TimedLlmClient(LlmClient delegate, MeterRegistry meters) { this.delegate = delegate;
this.meters = meters; }
    public String provider() { return delegate.provider(); }
    public String model() { return delegate.model(); }
    public LlmResponse complete(LlmRequest r) {
        return Timer.builder("llm.complete").tag("provider", provider()).register(meters)
                    .record(() -> delegate.complete(r));
    }
}
```

Proxy

Intent: a surrogate that **controls access** to another object — lazy loading, remote access, security, transactions, caching.

	Decorator	Proxy
Purpose	Add behaviour	Control access
Who creates the wrapper	The client composes it explicitly	Usually a framework, transparently
Knows the real object	Received via constructor	Often creates/locates it itself
Typical stacking	Many layers on purpose	One proxy with an interceptor chain

JDK: `java.lang.reflect.Proxy` (dynamic proxies), RMI stubs. Spring: **every AOP feature** (`@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize`), `@Configuration` CGLIB subclasses, scoped proxies, Spring Data repository interfaces (a proxy around `SimpleJpaRepository`), Hibernate lazy-loading proxies (`HibernateProxy`), Feign/HTTP interface clients. Chapter 16 covers the mechanics.

Facade

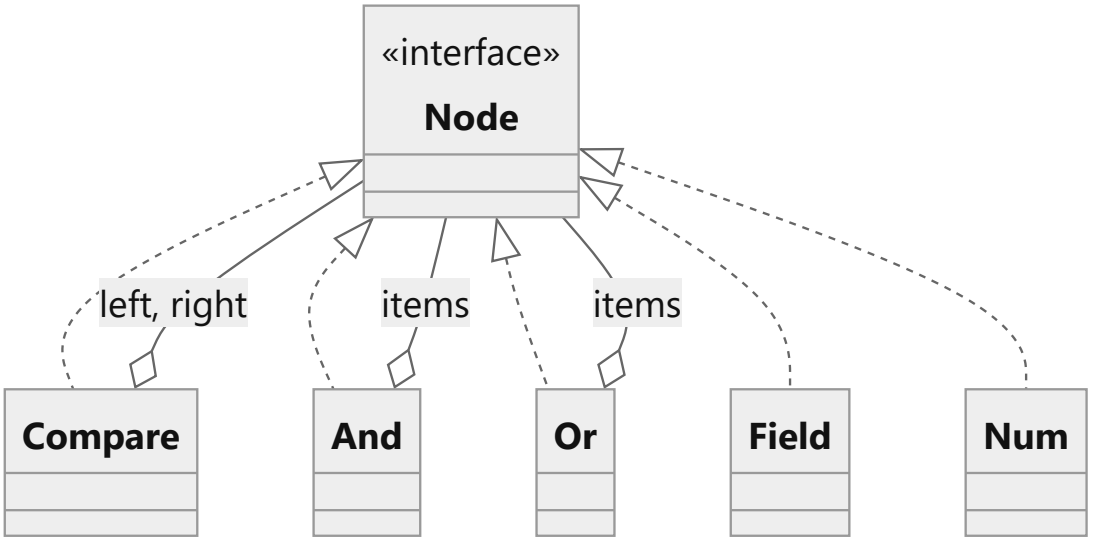
Intent: a simple interface to a complex subsystem.

`JdbcTemplate` is a facade over JDBC (connection from `DataSource`, statement, result set mapping, exception translation, resource closing). Also `RestTemplate` / `RestClient`, `KafkaTemplate`, SLF4J (facade over Logback/Log4j2), `javax.faces.context.FacesContext`.

> **RefinePoint:** `AuditTrail.record(orgId, action, entityType, entityId, details)` is a facade: behind one call it reads the current HTTP request from `RequestContextHolder`, takes the client IP from `X-Real-IP` (never the client-supplied `X-Forwarded-For`), resolves the user from the JWT, builds an `AuditEvent` and writes it to the outbox through `EventPublisher`. Services never touch any of that. `InternalClient` is the same idea for service-to-service HTTP (base URL, internal token, observation, error mapping).

Composite

Intent: treat individual objects and compositions uniformly in a tree.

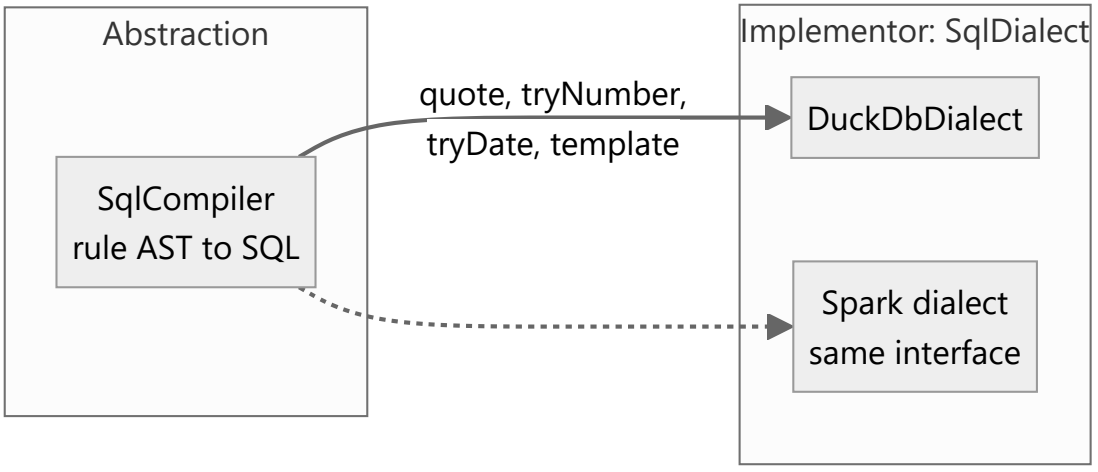


JDK/Spring: AWT `Container` / `Component`, `CompositePropertySource`, `CompositeCacheManager`, `HandlerMethodArgumentResolverComposite`, `CompositeMeterRegistry`, Jackson `JsonNode` trees.

> **RefinePoint:** a rule in `ryfn-expression` is a composite. `Node` is a `sealed interface`; leaves are `Field`, `Column`, `Text`, `Num`, `Bool`; composites are `And(List<Node>)`, `Or(List<Node>)`, `Compare(op, left, right)`, `Arith`, `If`, `Let`, `Call(fn, args)`. The evaluator treats `amount > 100 and party is "ACME"` and a single `amount > 100` exactly the same way — as a `Node`.

Bridge

Intent: decouple an abstraction from its implementation so the two can vary independently (two hierarchies connected by composition instead of an N×M class explosion).



JDK: **JDBC** (the `java.sql` API is the abstraction, each vendor `Driver` the implementor), JCA/ `java.security` providers, `java.util.logging` `Handler` / `Formatter`, SLF4J API vs bindings.

> **RefinePoint:** `SqlCompiler` turns a rule into SQL; everything engine-specific — identifier quoting, safe casts (`tryNumber` , `tryDate`), native function templates — goes through `SqlDialect` . New rule features change `SqlCompiler` ; a new engine adds a dialect. Neither multiplies the other.

Flyweight

Intent: share fine-grained objects to save memory; split **intrinsic** (shared, immutable) from **extrinsic** (passed in) state.

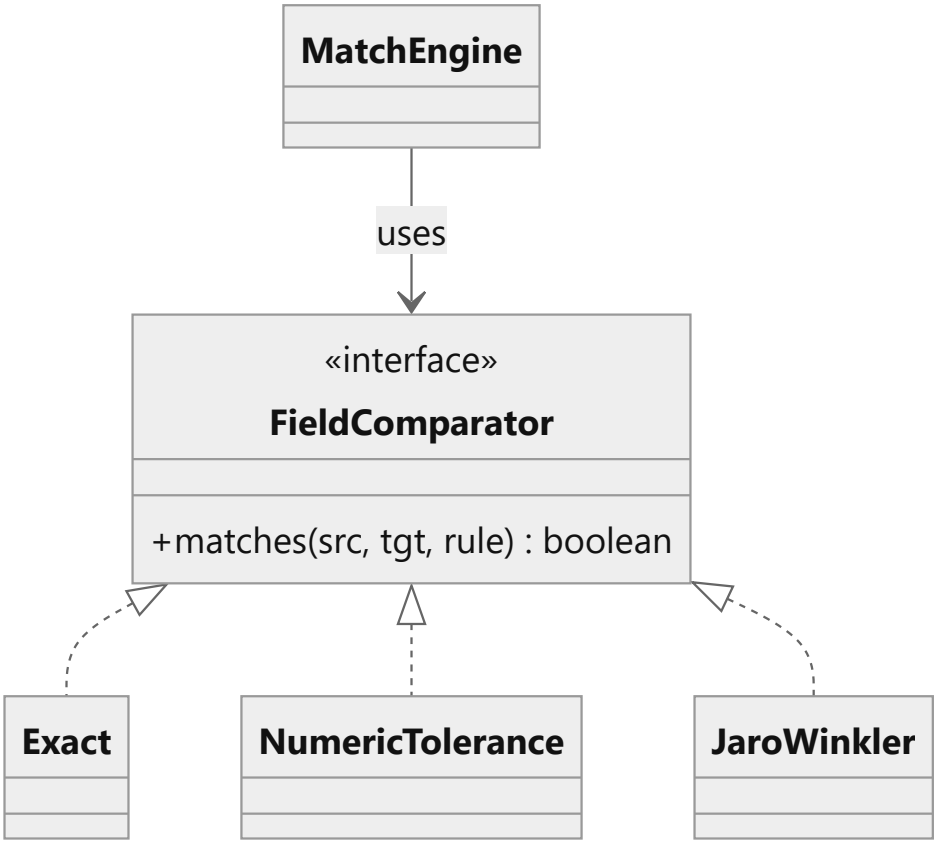
JDK: `Integer.valueOf` cache (-128..127), `Boolean.TRUE/FALSE` , `String` pool and `intern()` , `Character` / `Byte` / `Short` / `Long` caches, enum constants, `BigDecimal.ZERO/ONE/TEN` . Flyweights must be immutable.

> **RefinePoint:** the match engine shares the `MatchStrategy` enum constants (`EXACT` , `NUMERIC_TOLERANCE` , `JARO_WINKLER` , …) across millions of field comparisons; the per-comparison data (the two values, tolerance, date format) is extrinsic and comes from `MatchRuleConfig` .

13.4 Behavioral patterns

Strategy

Intent: define a family of interchangeable algorithms behind one interface; the client picks one at runtime.



Three ways to implement it in Java 21, from heaviest to lightest:

```
// 1. Interface + Spring beans, dispatch through an injected map (open set, plugins)
public interface ExportWriter { String format(); void write(RunResult r, OutputStream out); }
@Service class ExportService {
    private final Map<String, ExportWriter> writers;
    ExportService(List<ExportWriter> all) {
        writers = all.stream().collect(Collectors.toUnmodifiableMap(ExportWriter::format, w ->
w));
    }
    void export(String fmt, RunResult r, OutputStream out) {
        Optional.ofNullable(writers.get(fmt)).orElseThrow(() -> new BadRequestException("format "
+ fmt)).write(r, out);
    }
}

// 2. Lambdas — a strategy is just a function
BiPredicate<String, String> exact = Objects::equals;
BiPredicate<String, String> ci = String::equalsIgnoreCase;

// 3. Enum + exhaustive switch (closed set, config-driven)
boolean matched = switch (strategy) {
    case EXACT -> Objects.equals(src, tgt);
    case CASE_INSENSITIVE -> src.equalsIgnoreCase(tgt);
    case NUMERIC_TOLERANCE -> withinTolerance(src, tgt, rule.getTolerance());
    // ... every constant, compiler-checked
};
```

JDK/Spring: `Comparator`, `RejectedExecutionHandler`, `ThreadFactory`, `javax.crypto.Cipher` algorithms; Spring `PlatformTransactionManager` (JPA/JDBC/JTA), `ResourceLoader`, `CacheManager`, `PasswordEncoder`, `AuthenticationProvider`, `HandlerMethodArgumentResolver`.

> **RefinePoint:** a match rule names its algorithm in configuration: `MatchStrategy` is an enum (`EXACT`, `CASE_INSENSITIVE`, `NUMERIC_TOLERANCE`, `IGNORE_CHARS`, `DATE_ONLY`, `LOOKUP`, `CONTAINS`, `STARTS_WITH`, `JARO_WINKLER`, `LEVENSHTEIN`, `SOUNDEX`, `SIMILARITY`) stored with the rule, and `MatchEngine` dispatches with an exhaustive `switch`. We chose the enum form deliberately: the set is closed (users pick from a list in the designer), it serialises to a single column, and the compiler forces every new strategy to be handled. `LlmClient` is the other strategy in the system — chosen once at startup from `ai.provider`.

Template Method

Intent: a base class fixes the skeleton of an algorithm; subclasses fill in steps.

```
public abstract class AbstractImporter {
    public final ImportResult run(InputStream in) {          // the template — final
        var rows = read(in);
        var errors = validate(rows);
        if (!errors.isEmpty()) return ImportResult.rejected(errors);
        return ImportResult.ok(persist(rows));
    }
    protected abstract List<Row> read(InputStream in);       // primitive operation
    protected List<String> validate(List<Row> rows) { return List.of(); }  // hook with default
    protected abstract int persist(List<Row> rows);
}
```

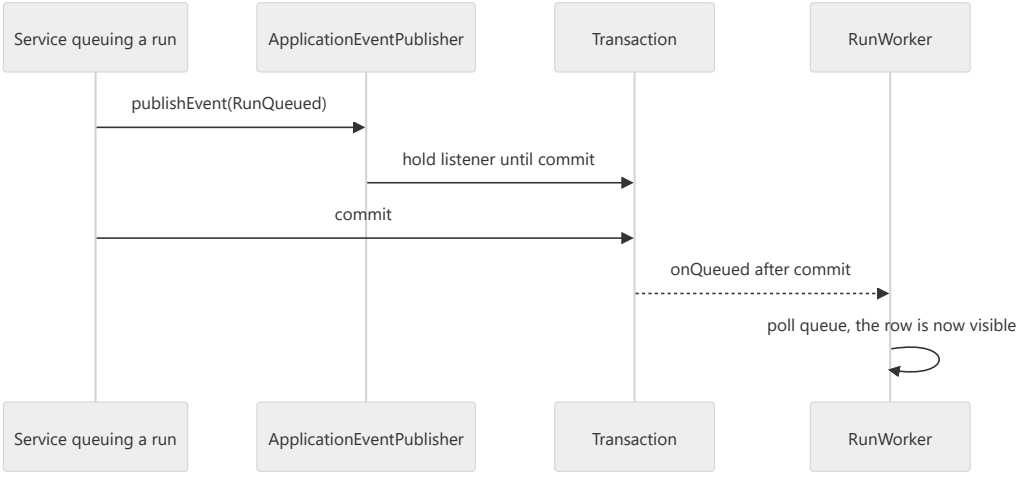
JDK/Spring: `AbstractList` (`get` / `size` are the steps), `HttpServlet.service()` → `doGet/doPost`, `InputStream.read(byte[])` built on `read()`; Spring's `OncePerRequestFilter.doFilterInternal`, `AbstractRoutingDataSource.determineCurrentLookupKey`, `AbstractMessageConverterMethodProcessor`.

Template + callback is the Spring flavour: instead of subclassing, pass the variable step as a lambda. `JdbcTemplate.query(sql, rowMapper)`, `TransactionTemplate.execute(status -> ...)`, `RestTemplate.execute(..., callback, extractor)`, `KafkaTemplate`. Composition beats inheritance, so the callback form is what you should write today.

> **RefinePoint:** `OutboxRelay.relay()` runs each batch inside `tx.execute(status -> sendBatch())` — `TransactionTemplate` owns begin/commit/rollback, we only supply the body. We needed programmatic transactions there because each batch must commit separately (a `@Transactional` on the loop would make one huge transaction).

Observer

Intent: a subject notifies registered observers when its state changes, without knowing who they are.

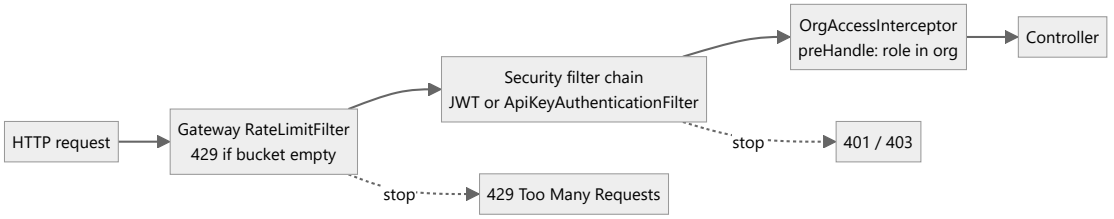


JDK: `java.util.Observer` (deprecated in 9 — not serializable, not thread-safe, `Observable` is a class), `PropertyChangeListener`, `java.util.concurrent.Flow` (reactive streams), Swing listeners. Spring: `ApplicationEventPublisher` + `@EventListener`, `@TransactionalEventListener`, `ApplicationListener`, `SmartApplicationListener`, context lifecycle events (`ContextRefreshedEvent`, `ApplicationReadyEvent`).

> **RefinePoint:** inside a service we use Spring events with `@TransactionalEventListener(phase = AFTER_COMMIT, fallbackExecution = true)` — `NotificationService` and `ChatDispatcher` in alert-notify, plus `WorkspaceSeats` in billing and `RunWorker` in run-service with the default phase — so a listener never acts on data that is later rolled back. `RunWorker.onQueued` is the diagram above: it wakes the poller only once the new run is committed and therefore visible. *Between* services the observer is distributed: the business transaction writes to `outbox_events`, `OutboxRelay` publishes to Kafka, and each consumer subscribes to a topic. The publisher still knows nothing about its subscribers — that is Observer at architecture scale (publish–subscribe).

Chain of Responsibility

Intent: pass a request along a chain of handlers; each handles it, passes it on, or stops it.



JDK/Servlet: `javax/jakarta.servlet.FilterChain` (`chain.doFilter(req, res)` to continue, return to stop), `java.util.logging.Logger` parent handlers, exception handling `try/catch` up the stack. Spring: **Spring Security's** `SecurityFilterChain` (~15 filters), MVC `HandlerInterceptor` chain (`preHandle` returning `false` stops), `HandlerExceptionResolver` chain, Spring Cloud Gateway `GatewayFilterChain`, AOP `MethodInterceptor` chain (`invocation.proceed()`).

> **RefinePoint:** a request to `/api/v1/organizations/{code}/...` goes through the gateway's `RateLimitFilter` (token bucket per IP and per caller, 429 when empty), then Spring Security (`KeycloakJwtConverter` for JWTs, `ApiKeyAuthenticationFilter` for automation keys), then `OrgAccessInterceptor.preHandle`, which resolves the org code from the path, checks the caller's role (GET needs VIEWER, writes need MEMBER) and checks workspace locks. Any link can end the request.

Command

Intent: encapsulate a request as an object so it can be queued, logged, retried, undone or executed elsewhere.

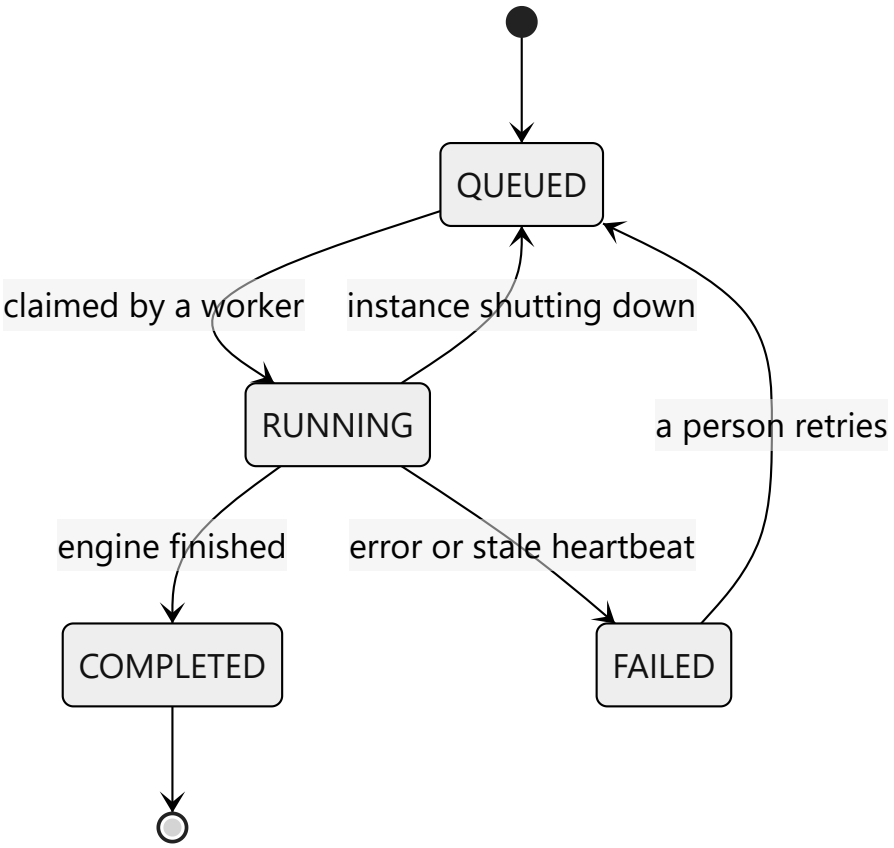
JDK: `Runnable`, `Callable` submitted to an `ExecutorService` (the executor is the invoker); `javax.swing.Action`. Spring: `TaskExecutor`, `@Async` turns a method call into a queued task, Spring Batch steps, `JdbcTemplate` callbacks.

```
public record RunCommand(UUID runId, UUID orgId, UUID processId, String triggeredBy) {}  // data-only command
```

> **RefinePoint:** our run queue is Command made durable. Starting a reconciliation does not run it — it saves a `recon_runs` row as `QUEUED`, and that row is the command. Any run-service instance claims the next row with `FOR UPDATE ... SKIP LOCKED` (priority first, then the workspace with fewer runs going, then the oldest) and executes it. Because the command is data in PostgreSQL it survives restarts, a person can retry it, and we scale by adding instances. The job-service `JobScheduler` and `ExportWorker` follow the same shape, and outbox rows are commands too ("send this event").

State

Intent: an object changes its behaviour when its internal state changes; each state owns the transitions allowed from it.




```
public enum RunStatus {
    QUEUED    { @Override Set<RunStatus> next() { return EnumSet.of(RUNNING); } },
    RUNNING   { @Override Set<RunStatus> next() { return EnumSet.of(COMPLETED, FAILED, QUEUED); }
},
    COMPLETED { @Override Set<RunStatus> next() { return EnumSet.noneOf(RunStatus.class); } },
    FAILED    { @Override Set<RunStatus> next() { return EnumSet.of(QUEUED); } };
    abstract Set<RunStatus> next();
    public RunStatus to(RunStatus target) {
        if (!next().contains(target)) throw new IllegalStateException(this + " -> " + target);
        return target;
    }
}
```

Classic GoF State uses one class per state with all operations (`start()` , `cancel()`) implemented per class; for small machines an enum with transition tables is clearer. JDK: `Thread.State` ; Spring: Spring Statemachine; also every `Iterator` has implicit state.

RefinePoint: a row in `recon_runs` follows exactly this machine, and because any number of instances work the same table, the transitions are SQL guarded by the current state. `RunWorker.claim()` selects only `status = 'QUEUED'` with `FOR UPDATE OF r SKIP LOCKED` . On graceful shutdown a `@PreDestroy` puts its own `RUNNING` rows back to `QUEUED` . A heartbeat reaper marks runs whose instance has been silent for two minutes as `FAILED` , and a person retries them. The enum above is the teaching form; ours is the same state machine expressed as guarded updates, which is the form that survives concurrent instances.

Visitor

Intent: add new operations to a stable object structure without changing its classes (double dispatch: `node.accept(visitor)` → `visitor.visitX(this)`).

```
// Classic GoF
interface NodeVisitor<R> { R visitCompare(Compare c); R visitAnd(And a); R visitField(Field f); }
interface Node { <R> R accept(NodeVisitor<R> v); }

// Java 21: sealed hierarchy + pattern-matching switch = visitor without accept()
static String toSql(Node n) {
    return switch (n) {
        case Field f -> quote(f.name());
        case Num x -> x.value().toPlainString();
        case Compare c -> toSql(c.left()) + op(c.op()) + toSql(c.right());
        case And a -> a.items().stream().map(X::toSql).collect(joining(" AND ", "(", ")"));
        // no default: the compiler proves every Node subtype is handled
    };
}
```

JDK: `java.nio.file.FileVisitor` (`Files.walkFileTree`), `javax.lang.model.element.ElementVisitor` (annotation processors), ASM `ClassVisitor`; Spring: `BeanDefinitionVisitor` (resolves placeholders in bean definitions).

> **RefinePoint:** the rule AST has several "visitors", each a pattern-matching `switch` over the sealed `Node`: `RuleEvaluator` compiles a node into a Java closure, `SqlCompiler` emits DuckDB SQL, `RuleWriter` prints the rule back in its English form, `RuleChecker` validates fields and types. Adding an operation is a new class; adding a node type breaks compilation in every switch until it is handled — exactly the trade-off we want for a language whose node set rarely changes.

Iterator

Intent: traverse a collection without exposing its representation.

External iterator: the client pulls (`Iterator.hasNext/next`). Internal iterator: the collection pushes to a callback (`forEach`, `Stream`). `Spliterator` adds splitting for parallel streams. Fail-fast iterators throw `ConcurrentModificationException` using `modCount`; fail-safe/weakly consistent iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) do not.

Spring: `Streamable`, `Page` / `Slice`, JPA `Stream<T>` repository methods (cursor-backed — must be closed, must run inside a transaction), `JdbcTemplate.queryForStream`, `RowCallbackHandler`.

> **RefinePoint:** `DataEngine.query(sql, Consumer<Map<String,Object>> row)` is an internal iterator: DuckDB pushes each result row to our callback, so writing a million-row export never materialises the result as a `List`.

Mediator

Intent: objects communicate through a central mediator instead of referencing each other (many-to-many becomes many-to-one).

Spring MVC's `DispatcherServlet` mediates between handler mappings, handler adapters, argument resolvers, message converters, exception resolvers and view resolvers — none of them know each other. `ApplicationEventPublisher` and a message broker are mediators too. JDK: `java.util.Timer` (mediates tasks), `ExecutorService`.

Mediator vs Observer: Observer is one-to-many broadcast; Mediator centralises *coordination logic* among peers.

Memento

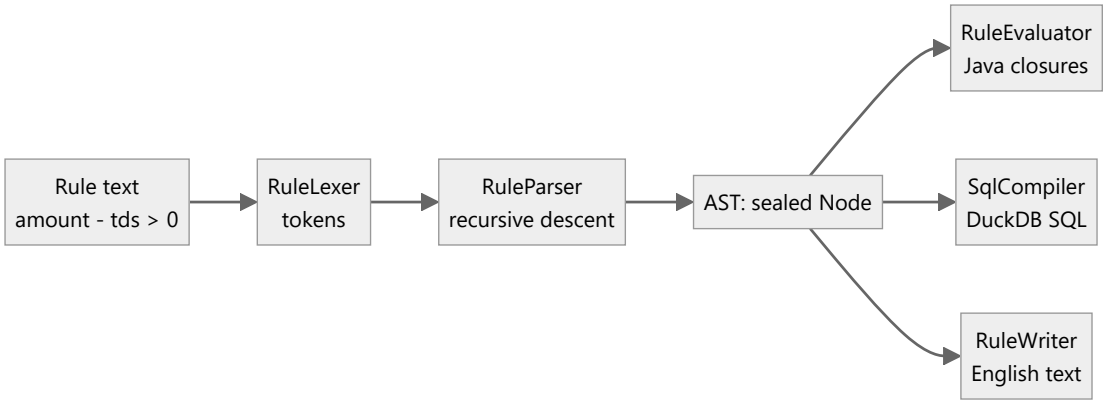
Intent: capture an object's internal state so it can be restored later, without breaking encapsulation.

Uses: undo, snapshots, optimistic "draft vs published". Java: serialization, records as immutable snapshots; Hibernate keeps a snapshot of each loaded entity's state to detect dirty fields (a memento used for comparison).

> **RefinePoint:** `ProcessSnapshot(orgId, process)` is read from reconciliation-service once when a run starts. The run executes against that snapshot, so a user editing the reconciliation mid-run does not change it half-way, and the result can always be explained against the exact definition it used.

Interpreter

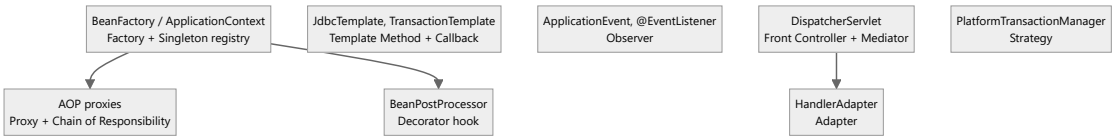
Intent: represent a grammar as classes and interpret sentences of the language by evaluating the AST.



JDK/Spring: `java.util.regex.Pattern` (compiles a regex into a node tree), `java.text.Format`, **SpEL** (`ExpressionParser` → AST of `SpelNode`), JPQL/HQL parsers, `@Scheduled` cron expressions, Spring Security's `@PreAuthorize("hasRole('ADMIN')")` (SpEL).

> **RefinePoint:** `ryfn-expression` is a full interpreter we wrote by hand: lexer → recursive-descent parser → AST of records implementing the sealed `Node` → evaluator. For large runs the same AST goes to `SqlCompiler`, which "interprets" by compilation — the whole rule becomes DuckDB SQL, with Java functions registered as UDFs for what SQL cannot express.

13.5 Patterns inside Spring — a map



Spring feature	Pattern
`BeanFactory.getBean`, `FactoryBean`, `@Bean`	Factory / Factory Method
Default bean scope	Singleton (registry, per container)
`@Transactional`, `@Cacheable`, Spring Data repositories	Proxy
`JdbcTemplate`, `RestTemplate`, `TransactionTemplate`	Template Method (+ callback), Facade
`ApplicationEventPublisher`	Observer
Filters, interceptors, `SecurityFilterChain`	Chain of Responsibility
`HandlerAdapter`, `MessageListenerAdapter`	Adapter
`DispatcherServlet`	Front Controller, Mediator
`PlatformTransactionManager`, `CacheManager`, `PasswordEncoder`	Strategy
`BeanDefinitionVisitor`	Visitor
SpEL	Interpreter
`ContentCachingRequestWrapper`, `TaskDecorator`	Decorator
`CompositePropertySource`, `CompositeCacheManager`	Composite
`UriComponentsBuilder`, `HttpSecurity`	Builder

13.6 Plugins with ServiceLoader — Factory + Strategy across jars

```
// ryfn-plugin-api
public interface DataConnector { ConnectorDescriptor descriptor(); ConnectorResult
fetch(ConnectorContext ctx); }

// a plugin jar contains META-INF/services/com.ryfnflow.plugin.connector.DataConnector
//   com.acme.SapConnector

// host
List<Plugins.Found<DataConnector>> connectors = Plugins.load(DataConnector.class);
```

RefinePoint: `Plugins.load(extensionPoint)` wraps `ServiceLoader.load(type, loader)` with a class loader over every jar in `RYFN_PLUGINS_DIR`. It iterates providers one by one and catches `ServiceConfigurationError` / `LinkageError` per plugin, so one broken jar is logged and left out instead of failing the service. Each result is a `Found<T>(plugin, source)` record so the UI can show which jar a connector came from. `DataConnector` and `FunctionPlugin` are the two extension points — Open/Closed at the deployment level: adding SAP support is dropping a jar, not editing core code.

13.7 Enterprise patterns every Spring developer uses

Pattern	Idea	In Spring / RefinePoint
Repository	Collection-like access to aggregates, hides persistence	Spring Data `JpaRepository`; domain-facing interfaces per aggregate
Unit of Work	Track changes in a business transaction, write them together	Hibernate persistence context: dirty checking, flush at commit
Identity Map	One in-memory object per row per unit of work	First-level cache: `em.find` twice returns the same instance
Data Mapper vs Active Record	Mapper keeps entities ignorant of DB; Active Record puts `save()` on the entity	JPA is a data mapper
DTO	Flat data carrier across a boundary	Records like `ReconRunDto`, `ProcessSnapshot`; never expose entities in APIs
Service Layer	Application operations and transaction boundary	`@Service` classes with `@Transactional`
Specification	Composable business predicates	Spring Data JPA `Specification<T>` (`where(a).and(b)`)
Null Object	A do-nothing implementation instead of `null`	`Collections.emptyList()`, `NoOpCacheManager`, `DataEngine.Rates.NONE`
Outbox	Write events in the same DB transaction as the change	`outbox_events` + `OutboxRelay`
Front Controller	One entry point dispatches all requests	`DispatcherServlet`, API gateway

```
// Specification: compose filters without a query per combination
static Specification<ReconBreak> inOrg(UUID org) { return (r, q, cb) -> cb.equal(r.get("orgId"),
org); }
static Specification<ReconBreak> olderThan(int days) {
    return (r, q, cb) -> cb.lessThan(r.get("createdAt"), Instant.now().minus(days,
ChronoUnit.DAYS));
}
breakRepo.findAll(inOrg(org).and(olderThan(30)), PageRequest.of(0, 50));
```

> **RefinePoint:** `DataEngine.Rates.NONE = (currency, on) -> null` is a null object: a run without currency conversion still gets a `Rates`, so the engine never null-checks the dependency itself — only the individual rate lookup.

13.8 Anti-patterns

Anti-pattern	Symptom	Fix
God object	3,000-line `ReconciliationService` with 25 dependencies	Extract classes by responsibility, facade for callers
Singleton abuse	Static global state, untestable code, hidden coupling	Spring singletons injected through constructors
Service Locator	`ctx.getBean(X.class)` inside business code	Constructor injection; `ObjectProvider` when lookup is really needed
Anaemic domain model	Entities are getters/setters, rules spread across services	Move invariants and transitions into the entity
Golden hammer / pattern-itis	`AbstractStrategyFactoryProvider` for one implementation	YAGNI — introduce a pattern on the second variation
Lava flow / dead code	Code nobody dares to delete	Tests + delete; version control remembers
Copy-paste programming	Same validation in 6 controllers	Extract, or move to an interceptor/aspect
Magic numbers/strings	`if (status == 3)`	Enums, constants, value objects
Primitive obsession	`String orgCode`, `BigDecimal amount` + `String ccy` everywhere	Value objects (`OrgCode` , `Money`)
Distributed monolith	Services that must deploy together, sync call chains	Bounded contexts, async events, own data
Shared mutable static state	`static SimpleDateFormat`	Immutable/thread-safe types (`DateTimeFormatter`)

13.9 Interview questions and answers

Q1. What is a design pattern, and why does it matter at senior level?

A named, proven solution to a recurring design problem in a given context. At senior level patterns matter as **vocabulary** (fast, precise design discussions) and as **trade-off catalogues** — each pattern buys flexibility at the price of indirection. Interviewers want to hear when you would *not* use one.

Q2. Classify the GoF patterns.

Creational (5): Singleton, Factory Method, Abstract Factory, Builder, Prototype. Structural (7): Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy. Behavioral (11): Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

Q3. Write a thread-safe lazy singleton. Which one would you use?

The holder idiom (lazy, no locking, JVM class-init guarantees safety) or the enum (also reflection- and serialization-proof). DCL works only with `volatile`. In a Spring application I would not write one at all — a singleton bean is injectable and testable.

Q4. Why does double-checked locking need `volatile`?

Without it, the write of the reference may be reordered before the constructor's field writes, so another thread can read a non-null reference to a partially constructed object. `volatile` creates a happens-before edge between the write and subsequent reads. Before Java 5 (old memory model) DCL was broken even with `volatile`.

Q5. How can a singleton be broken, and how do you protect it?

Reflection (`setAccessible(true)` on the private constructor) — guard in the constructor or use an enum. Serialization — implement `readResolve()` returning the instance. Cloning — don't implement `Cloneable`, or throw from `clone()`. Multiple class loaders — each loader has its own class; avoid by loading from a common parent. Distributed systems — one per JVM; use a DB lock or leader election if you truly need one cluster-wide.

Q6. Why is the enum singleton reflection-safe?

`Constructor.newInstance` explicitly checks `Modifier.ENUM` on the class and throws `IllegalArgumentException("Cannot reflectively create enum objects")`. Serialization writes only the constant's name; deserialization calls `Enum.valueOf`, returning the existing constant. Limitation: an enum cannot extend another class and is created eagerly at enum initialisation.

Q7. Is a Spring singleton bean the same as the Singleton pattern?

No. Spring guarantees one instance **per bean definition per container**, managed by the container's singleton registry (`DefaultSingletonBeanRegistry.singletonObjects`). You can define two beans of the same class, run two contexts, or call `new` yourself. It is a scope, not a class-level restriction — and that is a feature: it keeps the class testable.

Q8. Is a singleton an anti-pattern?

The GoF *global-access* singleton often is: hidden dependencies, global mutable state, hard to mock, ordering problems in tests. The *single-instance* idea is fine when the container manages it and you inject it. Stateless singletons are safe; stateful singletons need thread-safety.

Q9. Factory Method vs Abstract Factory?

Factory Method creates **one** product, with the concrete class decided by a subclass or a static method. Abstract Factory is an object that creates a **family** of related products that must be used together (engine + dialect, Connection + Statement). Abstract factories are often implemented with factory methods.

Q10. Static factory method vs constructor — pros and cons?

Pros: descriptive names (`Duration.ofSeconds`), may return cached instances (`Boolean.valueOf`), may return any subtype or a hidden implementation (`List.of` returns `ImmutableCollections.ListN`), can reduce generic verbosity. Cons: classes with only private constructors can't be subclassed, and static factories are harder to find in docs (naming conventions `of`, `from`, `valueOf`, `getInstance`, `newInstance`, `create` help).

Q11. When do you use Builder instead of constructors or setters?

Many parameters (especially optional ones), immutability requirements, validation before the object exists, and readability at call sites. Setters leave the object temporarily inconsistent and mutable; telescoping constructors are unreadable (`new Export(id, "csv", true, false, null, 3)`). For ≤4 required fields, a record is enough.

Q12. What does Lombok's `@Builder` generate, and what are its pitfalls?

A static nested `Builder` class, a `builder()` method, one setter-like method per field and `build()` calling an all-args constructor. Pitfalls: field initialisers are ignored unless `@Builder.Default`; no validation unless you write a custom `build()` or validate in the constructor; on JPA entities it conflicts with the no-args constructor requirement; `@Builder` + inheritance needs `@SuperBuilder`.

Q13. Prototype pattern — how in Java, and why not `clone()`?

Create objects by copying a configured instance. `clone()` is shallow, skips constructors, cannot assign `final` fields for deep copies and throws a checked exception. Use copy constructors, copy factories (`List.copyOf`) or record "withers". Spring's prototype scope is a different concept — a fresh instance per lookup.

Q14. Adapter vs Decorator vs Proxy vs Facade?

Adapter **changes** the interface to one the client expects. Decorator keeps the interface and **adds** behaviour, composed by the client, often stacked. Proxy keeps the interface and **controls access** (lazy, remote, security, transactions), usually created by a framework. Facade **simplifies** many interfaces into one new, higher-level interface.

Q15. Show the Decorator pattern in the JDK.

`new BufferedReader(new InputStreamReader(new GZIPInputStream(new FileInputStream(f)), UTF_8))` — each layer is a `Reader` / `InputStream` adding buffering, decoding or decompression. (`InputStreamReader` is technically an adapter from bytes to chars.) Also `Collections.unmodifiableList(list)`.

Q16. Where does Spring use the Adapter pattern?

`HandlerAdapter` lets `DispatcherServlet` invoke `@RequestMapping` methods, `HttpRequestHandler`s and old `Controller` implementations through one interface. Also `MessageListenerAdapter`, `WebMvcConfigurer` (formerly `WebMvcConfigurerAdapter`), and `AuthenticationProvider`s adapting LDAP/JWT/DB to Spring Security's model.

Q17. How did you use Adapter in your project?

"AI features talk to a provider-agnostic `LlmClient` interface — `complete(LlmRequest)` returning `LlmResponse`, with records for messages and attachments. `AnthropicClient` adapts it to Claude's Messages API, `OpenAiCompatibleClient` to our local GGUF model server. `AiConfig` picks one from `ai.provider`. Switching providers is a property change, and tests use a stub client."

Q18. Facade vs Service Layer?

A facade simplifies access to a subsystem; a service layer defines the application's use cases and transaction boundaries. A service often acts as a facade for controllers, but its primary job is orchestrating domain objects and repositories inside a transaction.

Q19. Explain Composite with an example from your work.

"Our rule language AST: `Node` is a sealed interface; leaves are fields, columns, text and number literals; composites like `And(List<Node>)`, `Or`, `Compare(op, left, right)`, `If` contain other nodes. Evaluation, SQL compilation and printing all recurse over the same tree and never care whether a node is a leaf or a composite."

Q20. What problem does Bridge solve? Give a JDK example.

It avoids an N×M class explosion when an abstraction and its implementation vary independently, by splitting them into two hierarchies joined by composition. JDBC is the classic: application code depends on `java.sql` interfaces, drivers implement them per database. In RefinePoint, `SqlCompiler` (what to compile) and `SqlDialect` (how each engine spells it) vary independently.

Q21. What is Flyweight? Where is it in the JDK?

Sharing immutable, fine-grained objects to save memory by separating intrinsic (shared) and extrinsic (passed-in) state. `Integer.valueOf` caches -128..127, `Boolean.valueOf`, the String pool, enum constants. That's why `Integer.valueOf(127) == Integer.valueOf(127)` is true and `128` is false.

Q22. Strategy vs State?

Same structure (context delegates to an interface), different intent. Strategy: the **client** chooses an algorithm, strategies don't know each other. State: the **object itself** switches implementation as its state changes, and states know their successors. If the implementation changes itself during the object's life, it is State.

Q23. Strategy vs Template Method?

Strategy uses composition: the whole algorithm is swappable at runtime. Template Method uses inheritance: the skeleton is fixed in a base class and only steps vary, chosen at compile time. Prefer Strategy (or template + callback) unless the skeleton genuinely must be enforced.

Q24. How do you implement Strategy in Spring without if/else?

Make each strategy a bean implementing a common interface with a key method, inject `List<Strategy>` and build a `Map` by key, or inject `Map<String, Strategy>` directly (keys are bean names). Unknown key → explicit error. For a closed set, an enum with an exhaustive `switch` or enum-per-constant behaviour is simpler.

Q25. Has Java 8+ made some patterns obsolete?

Lambdas collapse single-method Strategy, Command, Observer listeners and Template callbacks into functions. `Optional` covers some Null Object use. Records + sealed interfaces + pattern matching (Java 21) make Visitor mostly unnecessary. Streams are internal iterators. The *ideas* remain; the boilerplate disappears.

Q26. Explain Template Method in Spring's JdbcTemplate.

`JdbcTemplate` fixes the algorithm: get connection, create statement, set parameters, execute, iterate results, translate `SQLException` into `DataAccessException`, close everything. You supply only the variable part as callbacks — `RowMapper`, `PreparedStatementSetter`, `ResultSetExtractor`. It's template method via composition (callback) rather than subclassing.

Q27. Observer vs Publish–Subscribe?

Observer: the subject holds direct references to observers and calls them synchronously, in-process. Pub–sub: publishers and subscribers are decoupled by a broker/channel (Kafka, `ApplicationEventMulticaster`), may be asynchronous and in other processes. Spring events are in-process observer with a mediator-like multicaster; Kafka is pub–sub.

Q28. Why was `java.util.Observable` deprecated?

It is a class (forcing inheritance), not thread-safe in a useful way, not serializable, has a vague event model (`Object arg`) and an awkward `setChanged()` protocol. Replacements: `PropertyChangeSupport`, `Flow` (reactive streams), or a framework's event bus.

Q29. What happens if an `@EventListener` throws an exception?

With the default synchronous multicaster, the exception propagates to the publisher — it can roll back the publisher's transaction and other listeners after it may not run. With `@Async` listeners or a multicaster with a task executor, the exception goes to the `AsyncUncaughtExceptionHandler`/error handler instead. With `@TransactionalEventListener(AFTER_COMMIT)` the transaction is already committed, so the exception cannot roll it back.

Q30. How did you implement Observer reliably across services?

"Transactional outbox. The business change and an `outbox_events` row are written in one database transaction. `OutboxRelay` — `@Scheduled`, `SELECT ... FOR UPDATE SKIP LOCKED` in batches of 100 — sends rows to Kafka in sequence order and stops the batch on the first failure so order is kept. Consumers dedupe

with `ConsumedEvents.firstTime(consumer, eventId)`, an `INSERT ... ON CONFLICT DO NOTHING`. At-least-once delivery, effectively-once processing."

Q31. Chain of Responsibility in servlet filters — how does a filter stop the chain?

By not calling `chain.doFilter(request, response)` and writing a response itself (e.g., 401 or 429). Code before `doFilter` runs on the way in, code after it on the way out, so filters wrap each other like an onion.

Q32. Filter vs HandlerInterceptor vs AOP aspect?

Filters are servlet-level, see raw requests (before `DispatcherServlet`), and can wrap request/response — use for security, CORS, logging, compression. Interceptors are Spring MVC level, know the chosen handler and run `preHandle/postHandle/afterCompletion` — use for org access checks, locale. Aspects wrap bean methods anywhere, not just web — use for transactions, auditing, metrics.

Q33. Give a real Chain of Responsibility from your project.

"Gateway `RateLimitFilter` (token bucket per IP and per caller, 429), then Spring Security (Keycloak JWT or `ApiKeyAuthenticationFilter`), then `OrgAccessInterceptor.preHandle`, which extracts the org code from `/api/v1/organizations/{code}/...`, requires VIEWER for reads and MEMBER for writes, and checks workspace locks. Each step can end the request."

Q34. Command pattern — real uses?

Executor tasks (`Runnable` / `Callable`), undo/redo stacks, queued jobs, macro recording, CQRS commands. Durable commands are rows in a queue table: in our run-service a "run reconciliation" request is a `recon_runs` row that any worker later claims with `FOR UPDATE SKIP LOCKED`, which gives durability and horizontal scaling without a broker.

Q35. How do you implement undo with Command?

Each command implements `execute()` and `undo()` and stores what it needs to reverse itself (or a memento). The invoker keeps a stack of executed commands; undo pops and calls `undo()`. For persistent systems prefer compensating commands (a reversal entry) over destructive undo — accounting never deletes, it reverses.

Q36. What is double dispatch, and why does Visitor need it?

Java dispatches dynamically on the receiver only. Visitor needs the operation chosen by **both** the node type and the visitor type: `node.accept(visitor)` dispatches on the node, which then calls `visitor.visitCompare(this)`, dispatching on the visitor. Pattern-matching `switch` over a sealed hierarchy achieves the same with one explicit switch per operation.

Q37. When would you still use the classic Visitor in Java 21?

When the hierarchy is not sealed (third-party node types), when you need the visitor to be pluggable as an object (passing different visitors to a traversal framework like `Files.walkFileTree` or ASM), or for

compatibility with older Java. Otherwise sealed + switch is shorter and the compiler checks exhaustiveness.

Q38. What is the expression problem, and how do Visitor and Interpreter relate to it?

OO makes adding types easy and operations hard; Visitor/switch makes operations easy and types hard. Interpreter (methods on each node class: `node.evaluate()`) is the OO side; Visitor is the functional side. Our rule language chose the Visitor side because operations grow (evaluate, compile to SQL, print, check, explain) while node types are stable.

Q39. Explain Interpreter with your rule language.

"Users write rules like `amount - tds > 0 and party is not blank`. `RuleLexer` turns text into tokens, `RuleParser` (recursive descent with precedence levels) builds an AST of records implementing a sealed `Node`. `RuleEvaluator` compiles that tree into nested Java closures once and runs them per row; for big data `SqlCompiler` compiles it to DuckDB SQL instead. Functions come from a `FunctionCatalog`, plus `FunctionPlugin`s loaded with `ServiceLoader`."

Q40. Why compile an AST to closures instead of walking it per row?

Walking the tree per row repeats the `switch` dispatch, type checks and constant handling millions of times. Compiling once into a tree of lambdas (`env -> left.apply(env) > right.apply(env)`) resolves all structural decisions upfront; per row only the closures run, and the JIT can inline them. It's the same idea as Spring's SpEL compiler mode.

Q41. Iterator: fail-fast vs fail-safe?

Fail-fast iterators (`ArrayList`, `HashMap`) check `modCount` and throw `ConcurrentModificationException` if the collection changes structurally outside the iterator. Fail-safe/weakly consistent iterators (`CopyOnWriteArrayList` iterates a snapshot; `ConcurrentHashMap` traverses live but never throws) tolerate concurrent change. Fail-fast is best-effort, not a guarantee.

Q42. Mediator vs Facade?

Facade is one-directional: clients call the facade, subsystems don't know it exists. Mediator is bidirectional: colleagues talk *to* the mediator and the mediator talks back to them, coordinating their interaction. `DispatcherServlet` coordinates its strategies — mediator; `JdbcTemplate` hides JDBC — facade.

Q43. What is Memento, and where have you used snapshot thinking?

Capture state to restore or compare later without exposing internals. Hibernate's dirty checking keeps a loaded-state snapshot per entity. "Our runs execute against a `ProcessSnapshot` taken at run start, so edits during a run never change it and results are explainable against the exact definition used."

Q44. Which patterns does Spring Framework use? Name at least eight.

Factory (`BeanFactory` , `FactoryBean`), Singleton registry, Prototype (scope), Proxy (AOP, Spring Data, `@Configuration`), Template Method (`JdbcTemplate` , `AbstractRoutingDataSource`), Observer (`ApplicationEvent`), Chain of Responsibility (filters, interceptors, security chain), Adapter (`HandlerAdapter`), Strategy (`PlatformTransactionManager` , `Resource`), Front Controller (`DispatcherServlet`), Decorator (`BeanPostProcessor` wrappers, `TaskDecorator`), Builder (`UriComponentsBuilder`), Composite (`CompositePropertySource`), Visitor (`BeanDefinitionVisitor`), Interpreter (SpEL).

Q45. Which pattern is `BeanPostProcessor` ?

It's a hook (template method of the container lifecycle) that frequently acts as a **decorator/proxy factory**: `postProcessAfterInitialization` may return a different object wrapping the bean. `AbstractAutoProxyCreator` is a `BeanPostProcessor` that returns AOP proxies; that's how `@Transactional` beans become proxies.

Q46. Repository vs DAO?

DAO is persistence-oriented: one per table, CRUD and query methods mirroring the database. Repository is domain-oriented: a collection-like abstraction for **aggregates** (`save(run)` , `findOpenBreaks(org)`), part of the domain language. In Spring Data the line blurs — `JpaRepository` is both — but in DDD you keep one repository per aggregate root, not per table.

Q47. What is Unit of Work, and how does Hibernate implement it?

A unit of work tracks objects loaded and changed in a business transaction and writes all changes together. Hibernate's persistence context holds managed entities plus snapshots; at flush (before commit, before some queries, or explicit) it compares each entity with its snapshot and issues the `UPDATE` s — you never call `save` on a managed entity. Insert ordering, batching and the first-level cache (identity map) come with it.

Q48. Specification pattern in Spring Data?

`Specification<T>` is a functional interface `(root, query, cb) -> Predicate` . Specifications compose with `and` , `or` , `not` , `where` , so dynamic search screens become small reusable predicates instead of a repository method per filter combination. Repository must extend `JpaSpecificationExecutor<T>` .

Q49. What is the Null Object pattern? Is `Optional` a null object?

A do-nothing implementation of an interface used instead of `null` so callers don't branch: `Collections.emptyList()` , `NoOpCacheManager` , a `NoopNotifier` . `Optional` is different — it *signals* absence and forces the caller to decide; a null object *hides* absence behind default behaviour. Use null objects when "do nothing" is genuinely the right default.

Q50. What is DTO and why not return entities from controllers?

A DTO is a boundary-specific data carrier. Returning entities couples API to schema, leaks fields (password hashes, internal IDs), triggers lazy-loading exceptions or N+1 queries during serialization, creates JSON recursion on bidirectional relations, and lets clients mass-assign fields on input. Records make DTOs nearly free.

Q51. Service Locator vs Dependency Injection?

Service Locator: code *asks* a registry for dependencies (`ctx.getBean(...)`) — dependencies are hidden, tests need a configured registry. DI: dependencies are *given* through the constructor — visible, compile-checked, trivially mocked. Spring's `ObjectProvider` is a scoped, typed lookup that is acceptable for optional or lazy dependencies.

Q52. How does ServiceLoader work, and how did you use it?

`ServiceLoader.load(Interface.class, classLoader)` reads every `META-INF/services/<fully.qualified.Interface>` file on the class path, instantiates the listed classes lazily (public no-arg constructor, or a `provider()` static method for modules) and iterates them. "Our `Plugins.load` uses it with a class loader over the plugins folder, catches `ServiceConfigurationError` per provider so a broken jar can't take the service down, and returns `Found(plugin, source)` records."

Q53. Name common anti-patterns you've removed from a codebase.

God classes (split by responsibility), service locator lookups (replaced by constructor injection), anaemic entities with status setters (moved transitions into the entity), static mutable caches (replaced by bounded caches with TTL), copy-pasted tenant checks in every controller (moved to one interceptor), magic status codes (enums), distributed-monolith synchronous chains (events).

Q54. What is "pattern-itis" and how do you avoid it?

Applying patterns for their own sake: one-implementation interfaces, factories for objects with one constructor, abstract base classes with a single child. Avoid by following YAGNI, introducing an abstraction when the second variation appears, and asking "what change does this make cheaper?" If you can't name the change, remove the indirection.

Q55. Design question: how would you make export formats extensible?

Strategy: `ExportWriter` interface with `format()` and `write(result, out)` ; one bean per format (CSV, XLSX, PDF); a service builds a `Map<String, ExportWriter>` from the injected list; unknown formats → 400. Rows streamed through an internal iterator so memory stays flat. New format = new class, no edits elsewhere (OCP). Add a decorator for metrics or compression if needed.

Q56. Design question: a payment provider integration that may change.

Adapter + anti-corruption layer: define our own `BillingProvider` port in domain terms (plans, subscriptions, seats); implement a Paddle adapter that translates Paddle's API and webhook payloads into our events; keep Paddle IDs at the edge. Webhook handlers are idempotent (event id dedupe). Switching or adding a provider (e.g., marketplace billing) becomes another adapter.

Q57. Which pattern would you use to add retries, timeouts and metrics around an HTTP client?

Decorator (or AOP proxy) — each concern a wrapper with the same interface: `Timed(Retrying(TimeLimited(Real)))`. Order matters: metrics outermost to see total latency including retries, or innermost to see per-attempt latency. In Spring, Resilience4j annotations implement the same thing as proxy interceptors.

Q58. Builder vs Abstract Factory?

Builder constructs **one complex object step by step** and returns it at the end; Abstract Factory returns **several related objects**, each immediately. A builder can use an abstract factory to produce its parts.

14. Clean Code, DDD & Hexagonal Architecture

These are the questions that separate a strong coder from a lead. There is rarely one right answer, so the interviewer scores your reasoning: which boundary you drew, what it cost, and when you would have chosen differently. Definitions alone will not carry you. Have a real boundary from your own system ready for every concept, plus one place where you decided the pattern was not worth it.

14.1 Clean code rules that still matter at senior level

Most clean-code advice is about reducing the cost of the next change. Measure each rule against that, not against a checklist.

Rule	What it means in practice	Where seniors bend it
Names reveal intent	`unmatchedBreaks`, not `list2`. Name by role, not type	Short names are fine in tiny scopes (`i`, `e`, `r` in a lambda)
Functions do one thing	One level of abstraction per method	A 60-line linear method can beat 12 two-line helpers you must jump through
Few parameters	Three or more related values become a record	Do not invent a parameter object for one call site
No side effects in queries	`isValid()` must not save anything	Caching inside a query is an acceptable hidden effect
Comments explain why	Why this order, why this limit, what bug this prevents	Public library contracts deserve full docs
Fail early, fail loudly	Validate at the boundary, then trust internal data	Do not re-validate at every layer
Prefer immutability	Records, `final`, `List.copyOf`	Performance-critical buffers can be mutable if contained
Errors are part of the design	Domain exceptions, one mapping point to HTTP	Do not catch what you cannot handle
Small public surface	Package-private by default, `public` by decision	Framework needs sometimes force `public`
Tests describe behaviour	Test names read as rules, assert outcomes	Do not test private details

```
// Hard to change: the reader has to simulate the code to understand the rule
if (r.getS() == 2 && (u.getRole() == 1 || u.getRole() == 3) && r.getD() != null) { approve(r); }

// Intent first: the condition has a name and a home
if (run.isAwaitingApproval() && approver.canApprove(run)) { run.approveBy(approver); }
```

> **RefinePoint:** `OrgAccessInterceptor.requiredRole` is deliberately boring: a `switch` from HTTP method to role (`GET` , `HEAD` need VIEWER, everything else needs MEMBER), one rule per line. Security-relevant code should be the easiest code in the system to read, because every future reviewer must be able to see that it is correct.

Comments and error handling

A good comment records what the code cannot say. From our real code: `AuditTrail` notes that `X-Real-IP` is set by nginx and `X-Forwarded-For` is client-supplied and never read. Delete that comment and someone will "fix" it one day and let clients forge their audit address.

For errors, seniors keep three rules. Throw domain-meaningful exceptions from the domain and service layer. Convert them once at the edge, in a `@RestControllerAdvice` . Never swallow, and never log and rethrow the same exception, because that produces duplicate stack traces.

```
// ryfn-platform GlobalExceptionHandler is the one place that turns exceptions into responses
@ExceptionHandler(PlanLimitExceededException.class)
ResponseEntity<ApiResponse<Void>> planLimit(PlanLimitExceededException e) {
    return
ResponseEntity.status(HttpStatus.PAYMENT_REQUIRED).body(ApiResponse.error(e.getMessage()));
}
```

A service that hits a plan limit throws and knows nothing about HTTP; only the handler knows that this becomes a 402.

14.2 Code smells and their refactorings

Smell	Symptom	Refactoring
Long method	You scroll, and comments mark sections	Extract Method, then Extract Class if state clusters
Large class / God class	2,000 lines, 20 dependencies	Extract Class along clusters of fields and methods
Long parameter list	`create(a, b, c, d, e, f)`	Introduce Parameter Object, a record
Primitive obsession	`String iban`, `BigDecimal amount` + `String ccy`	Replace Primitive with Value Object
Feature envy	A method uses another class's data more than its own	Move Method to the data
Data clumps	The same three fields travel together	Extract Class, usually a value object
Shotgun surgery	One change touches 15 files	Move Field/Method to one place
Divergent change	One class changes for many unrelated reasons	Extract Class (SRP)
Switch on type repeated	The same `switch` in five places	Polymorphism, or a sealed type with one exhaustive switch per operation
Speculative generality	Abstract factory with one product	Collapse Hierarchy, Inline Class
Temporal coupling	`init()` must be called before `run()`	Constructor establishes the invariant, or a builder
Message chains	`a.getB().getC().getD()`	Hide Delegate, Tell-Don't-Ask
Dead code / lava flow	Nobody knows if it runs	Delete it; git remembers
Boolean flag argument	`export(run, true, false)`	Two methods, or an enum

How to refactor safely: characterisation tests first (pin current behaviour, including its bugs), small behaviour-preserving steps, separate commits for refactor and behaviour change, and use IDE automated refactorings because they do not make typos. For big moves use **strangler fig** or **branch by abstraction**: put an interface in front of the old code, build the new one behind it, switch, delete the old.

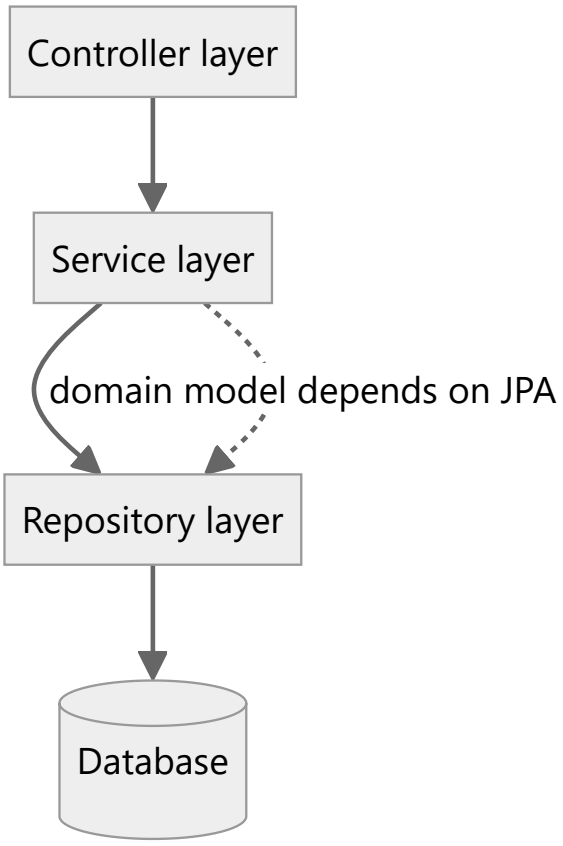
```
// Before: feature envy, the service reaches into the run's data
if (run.getStatus().equals("RUNNING") && Duration.between(run.getHeartbeatAt(), now).toMinutes()
>= 2) { fail(run); }

// After: the question moves to the object that owns the data
if (run.isStale(now, Duration.ofMinutes(2))) { fail(run); }
```

14.3 Layered vs hexagonal vs clean/onion

Layered (N-tier)

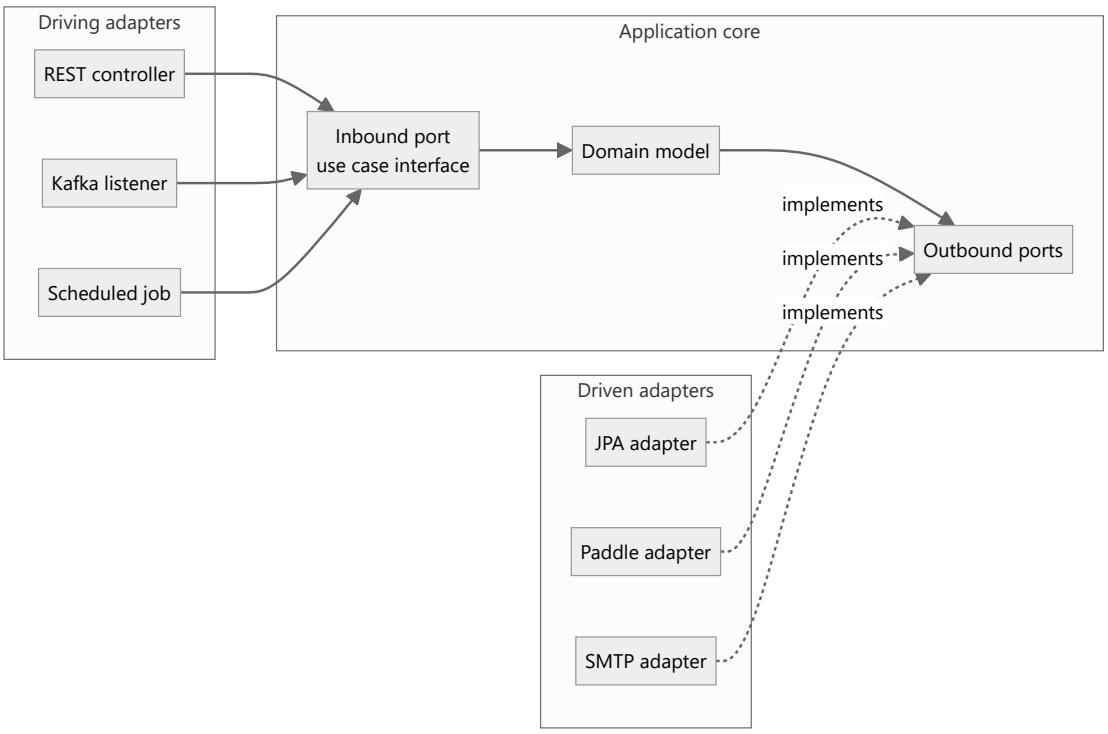
Controllers call services, services call repositories, and dependencies point downward to the database.



It is simple, universally understood, and the right default for CRUD. Its weakness is that the domain ends up depending on infrastructure: entities carry JPA annotations, services know about Kafka and HTTP clients, and business logic is hard to test without Spring and a database. **Strict layering** allows a layer to call only the one below; **relaxed layering** allows skipping, which is what most teams actually do.

Hexagonal (ports and adapters)

Alistair Cockburn's idea is that the application sits in the middle and everything else, including the UI, is an external actor plugged in through a port.



The key rule is the **direction of dependencies**: everything points inward. An outbound port is an interface owned by the core and implemented outside it, which is the Dependency Inversion Principle applied at architecture scale.

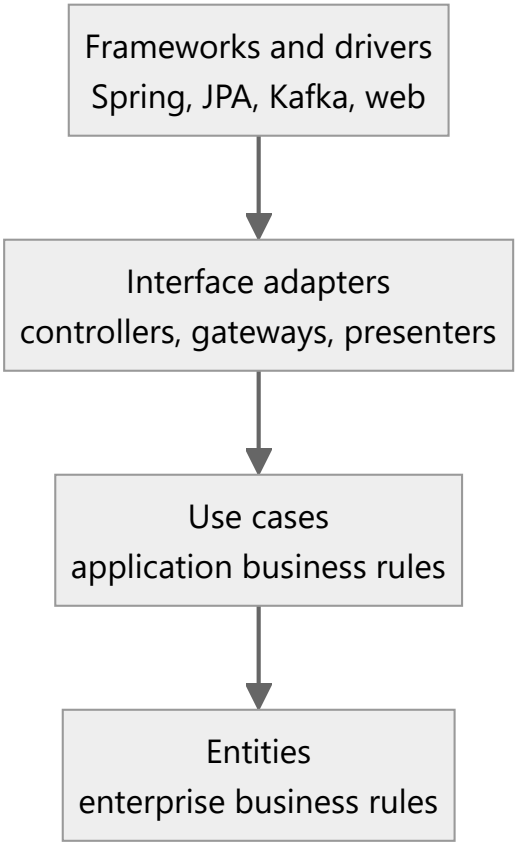
Term	Meaning	Example
Inbound (driving) port	What the outside can ask the app to do	`StartRunUseCase`
Inbound adapter	Translates an external trigger into a port call	`RunController`, Kafka listener
Outbound (driven) port	What the app needs from the outside	`LlmClient`, `Mailer`, `PlanLimits`
Outbound adapter	Implements the port using technology	`AnthropicClient`, `EmailSender`

```
// Core: owned by the domain, no framework imports
public interface PlanLimits {
    void assertCanAddReconProcess(UUID orgId);
    void assertSeatAvailable(UUID orgId, long seatsInUse);
    void assertCanReconcile(UUID orgId);
}
// Adapter in each service: calls billing over InternalClient, cached in a TtlCache.
// The billing service implements the same port from its own tables.
```

RefinePoint: we do not practise pure hexagonal in every service, but the ports that matter are real. `PlanLimits` in `ryfn-platform` is commented exactly so: "Modules call this before creating something that a plan restricts and never reach into billing." `Mailer` is the same idea for email, and `EmailSender` is its implementation in alert-notify. `LlmClient` is the outbound port in ai-service. `DataEngine` hides DuckDB. We drew ports where the technology or the owning team could change, and left CRUD screens as plain controller, service and repository.

Clean and onion architecture

Robert Martin's Clean Architecture and Jeffrey Palermo's Onion are the same idea as hexagonal with named rings. The **Dependency Rule** says source-code dependencies only point inward.



	Layered	Hexagonal	Clean / Onion
Core idea	Horizontal slices	Inside vs outside, through ports	Concentric rings, dependency rule
Domain depends on DB?	Usually yes	No	No
Test business logic without Spring	Hard	Easy	Easy
Boilerplate	Low	Medium	Medium to high
Best for	CRUD, small services	Rich domain, many integrations	Large, long-lived domains
Risk	Domain logic leaks into services	Ports with one implementation everywhere	Mapping layers with no behaviour

Trap: "hexagonal means no annotations in the domain" is often taken too far. Pure domain classes plus separate JPA entities plus mappers can triple the code for a simple feature. Many teams keep JPA annotations on domain entities as a pragmatic compromise and accept that coupling. Say you know the trade-off, and that you apply the purity where the domain is genuinely complex.

14.4 Package by layer vs package by feature

```
Package by layer                    Package by feature
com.app.controller                  com.app.exports
  RunController                       ExportController
  ExportController                    ExportService
com.app.service                       ExportWorker       (package-private)
  RunService                          ExportFile
  ExportService                     com.app.runqueue
com.app.repository                    RunWorker
  RunRepository                       RunQueueService
  ExportRepository                  com.app.breaks
                                      BreakController
                                      BreakService
```

	By layer	By feature
Cohesion	Low: one change touches every package	High: one feature in one folder
Encapsulation	Everything must be `public` to cross packages	Package-private works, and the compiler guards the boundary
Finding code	Find the layer, then search	Find the feature, and everything is there
Splitting out a service	Painful	A feature package is already a candidate
Familiarity	Very high	High

RefinePoint: run-service is package-by-feature: `runqueue`, `exports`, `breaks`, `journal`, `openitems`, `retention`, `archive`, `workflow`. Identity is the same: `apikey`, `mfa`, `sso`, `group`, `organization`. Within a large feature, such as `reconciliation`, a second level of `controller/service/entity/repository` appears, which is a pragmatic hybrid. When a person asks "where is export retry?", the answer is one folder.

14.5 DDD strategic design

Strategic design decides **where the boundaries are**, and it is where most of DDD's value sits. Tactical patterns are worth little inside a badly drawn boundary.

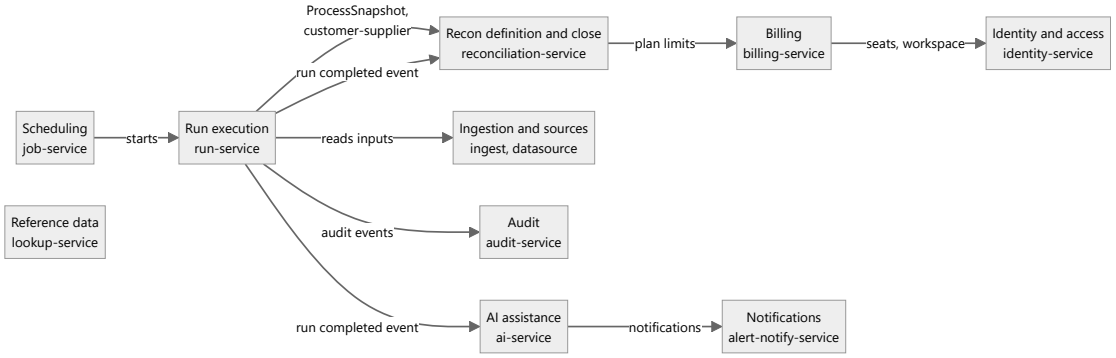
Ubiquitous language: one vocabulary shared by domain experts and code, valid **inside one bounded context**. In reconciliation that means "break" (an unmatched or disputed item), "pass" (one round of multi-pass matching), "tolerance", "close". If finance says "break", no class should be called `UnmatchedRecordException`.

Subdomains describe the problem space:

Type	Meaning	RefinePoint	Strategy
Core	Why customers pay, competitive edge	Reconciliation design, matching engine, rule language, AI drafting	Build it, best design effort, rich model
Supporting	Necessary, specific to us, no edge	Ingest, data sources, lookups, jobs, reports, alerts	Build simply
Generic	Solved problem	Authentication, payments	Buy: Keycloak, Paddle

Bounded context: an explicit boundary inside which a model and its language are consistent. The same real-world word can legitimately mean different things across contexts, and that is the point. "Reconciliation" in the design context is a definition with field mappings and rules; in the run context it is an execution with a status and a result table; in billing it is merely a counted unit of usage against a plan.

RefinePoint's context map



Context	Service(s)	Owns
Identity and access	identity-service, with Keycloak	Workspaces, members, roles, API keys, SSO, MFA
Recon definition	reconciliation-service	Reconciliation processes, field mappings, computed fields, close
Run execution	run-service	Queue, runs, results, breaks, exports, journals
Scheduling	job-service	Schedules and jobs
Ingestion	ingest-service	SFTP inbox, inbound email, pulls from APIs
Data sources	datasource-service	Connections, connectors, refresh
Reference data	lookup-service	Lookup tables, workspace rule functions
Reporting	report-service	Reports
Audit	audit-service	Immutable record of actions
Billing	billing-service	Subscriptions, invoices, license keys
Notifications	alert-notify-service	Alert rules, email, webhooks, chat
Instant recon	instant-recon-service	One-off reconciliations for marketplace apps
AI assistance	ai-service	Drafts, explanations, anomaly detection, digests

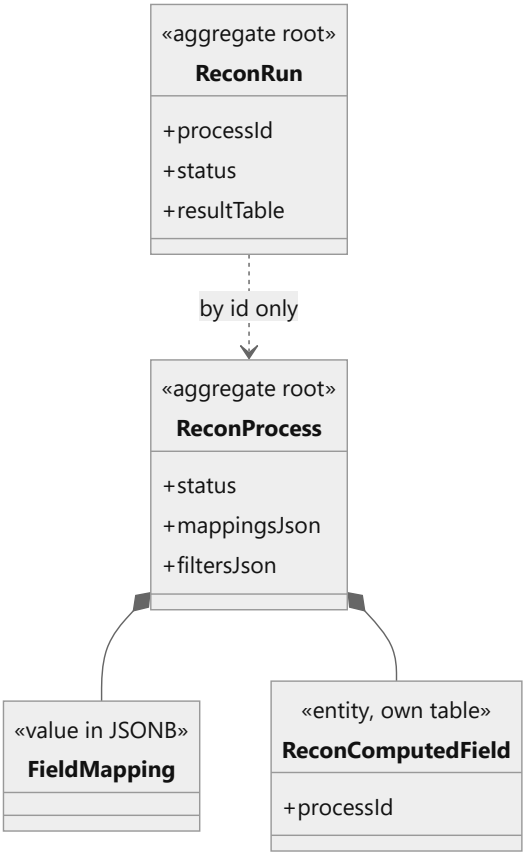
The api-gateway is not a context. It holds no domain model, only routing and rate limiting, which is why there are 13 business services behind it.

Context-map relationships you must be able to name:

Relationship	Meaning	Example
Shared Kernel	Two contexts share a small model and must coordinate changes	`ryfn-platform`, `ryfn-engine`
Customer–Supplier	Upstream plans around the downstream's needs	run-service reads `ProcessSnapshot` from reconciliation-service
Conformist	Downstream simply accepts the upstream's model	A consumer taking a vendor's format as is
Anti-Corruption Layer	Downstream translates to protect its own model	Billing's Paddle and marketplace code
Open Host Service	Upstream offers a documented protocol to everyone	REST APIs, the MCP server
Published Language	A well-defined interchange format	Event envelopes on Kafka topics, `ProcessSnapshot`
Separate Ways	No integration at all, duplicate instead	Each service keeps its own copy of what it needs
Partnership	Two teams succeed or fail together	Rarely relevant for a solo builder

Trap: a shared library is a shared kernel, and its cost is real. After editing `ryfn-platform` we must rebuild every service, or the unchanged ones keep the old jar. That is the price of a shared kernel: keep it small, stable and technical, and keep domain rules out of it where you can.

14.6 DDD tactical design



Building block	Defined by	Rule
Entity	Identity that persists through change	`equals` on id; behaviour lives on it
Value object	Its attributes	Immutable, self-validating, replaced rather than changed
Aggregate	A consistency boundary	Changed as one, in one transaction
Aggregate root	The only door into the aggregate	Outsiders hold only its id
Domain event	Something that happened, in past tense	Immutable, `RunCompleted` not `CompleteRun`
Repository	Collection-like access to aggregates	One per root; the interface belongs to the domain
Domain service	Domain logic that belongs to no single object	Stateless, named in the domain language
Application service	Orchestrates a use case	Transactions, security, calls domain; holds no rules
Factory	Builds a complex aggregate validly	Static method or a class

Value objects: the cheapest big win

```
public record Money(BigDecimal amount, Currency currency) implements Comparable<Money> {
    public Money {
        Objects.requireNonNull(amount); Objects.requireNonNull(currency);
        amount = amount.setScale(currency.getDefaultFractionDigits(), RoundingMode.HALF_EVEN);
```

```
    }
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("Currency
mismatch");
        return new Money(amount.add(other.amount), currency);
    }
    public boolean within(Money other, BigDecimal tolerance) {
        return currency.equals(other.currency) &&
amount.subtract(other.amount).abs().compareTo(tolerance) <= 0;
    }
    @Override public int compareTo(Money o) { return amount.compareTo(o.amount); }
}
```

An invalid `Money` cannot exist, equality is by value, and `BigDecimal`'s scale-sensitive `equals` trap is contained in one place. Persist it with `@Embeddable`, or with an `AttributeConverter` for one-column values.

Entity and aggregate root with behaviour

```
@Entity
public class Invoice extends BaseOrgEntity {
    @Enumerated(EnumType.STRING) private InvoiceStatus status = InvoiceStatus.DRAFT;
    @OneToMany(mappedBy = "invoice", cascade = CascadeType.ALL, orphanRemoval = true)
    private final List<InvoiceLine> lines = new ArrayList<>();

    public void addLine(String description, Money amount) {
        if (status != InvoiceStatus.DRAFT) throw new IllegalStateException("Invoice is " +
status);
        lines.add(new InvoiceLine(this, description, amount));
    }
    public void issue(Clock clock) {
        if (lines.isEmpty()) throw new IllegalStateException("Cannot issue an empty invoice");
        status = InvoiceStatus.ISSUED;
        registerEvent(new InvoiceIssued(getId(), clock.instant()));   // or AbstractAggregateRoot
    }
    public List<InvoiceLine> lines() { return List.copyOf(lines); }   // no mutable list leaks
}
```

Outsiders cannot add a line without going through the root, so "no edits after issue" holds no matter which service calls it. Spring Data's `AbstractAggregateRoot` provides `registerEvent`, and the events are published when `repository.save(...)` is called.

Domain service vs application service

```
// Domain service: a rule spanning two records, no I/O, no transaction
public final class ToleranceMatcher {
    public boolean matches(Rec source, Rec target, MatchRuleConfig rule) { /* ... */ return true;
}
}

// Application service, the general shape: the use case, the transaction, the wiring, no business
rules
@Service @RequiredArgsConstructor
class InvoiceIssuing {
    private final InvoiceRepository invoices;
    private final PlanLimits limits;
    private final AuditTrail audit;
    private final Clock clock;
```

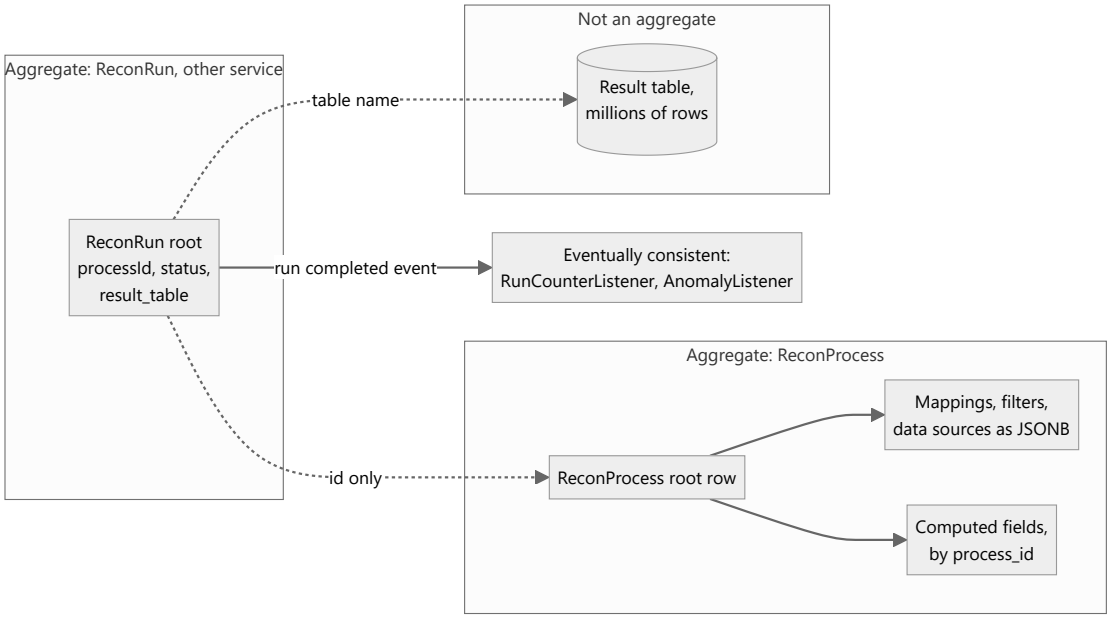
```
    @Transactional
    public void issue(UUID orgId, UUID invoiceId) {
        limits.assertFeatureEnabled(orgId, "invoices");
        Invoice invoice = invoices.findByIdAndOrgId(invoiceId, orgId).orElseThrow();
        invoice.issue(clock);                                    // the rule lives on the
aggregate
        audit.record(orgId, "INVOICE_ISSUED", "Invoice", invoiceId, Map.of());
    }
}
```

RefinePoint: we are honest about the spectrum. Our core rule language and match engine are rich and framework-free: `Node`, `RuleEvaluator`, `ReconEngine` and `PassMatcher` import no Spring or JPA. Many entities, `ReconRun` among them with its `status` string, are closer to data holders, with the transitions enforced by guarded SQL because many instances work the same rows. For a queue that is the right tool: a database claim is the invariant, and an entity method could not enforce it across processes anyway. `ReconProcess` shows another practical choice: mappings, filters and data sources are JSONB columns on the root row, because a definition is always read and written whole, so the aggregate boundary is literally one row. Computed fields, which have their own table, reference it by `process_id`.

14.7 Aggregate sizing and consistency boundaries

Vaughn Vernon's four rules:

1. **Protect true invariants inside the boundary.** Only rules that must hold at every commit go in the aggregate.
2. **Design small aggregates.** Large ones lock more, load more, and conflict more.
3. **Reference other aggregates by id,** never by object reference.
4. **Use eventual consistency outside the boundary,** through domain events.



The sizing test: ask "if this rule were violated for two seconds, would anyone be harmed?" If not, it does not need to live inside one aggregate. A monthly invoice total staying exactly equal to the sum of its lines is a true invariant. A dashboard counter being two seconds stale is not.

Concurrency: one transaction changes one aggregate (the guideline, not a law). Protect the aggregate with `@Version` optimistic locking on the root, and bump the root's version when a child changes. A big aggregate that many users edit gets constant `OptimisticLockException`s, which is the classic sign that it is too large.

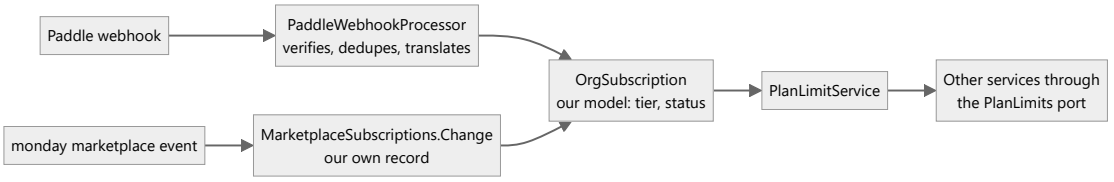
> **RefinePoint:** a run's results can never be part of its aggregate. A run with millions of result rows lives in its own table (`result_table` on `recon_runs`), read through `ResultQueries` . And run-service holds the reconciliation only as a `ProcessSnapshot` , taken once at start: an id plus a read-only copy, never a live object. That one decision removed the "what if someone edits the definition during a run" problem completely. The denormalised "total runs" and "last run" on the process are updated eventually by `RunCounterListener` from the run-completed topic: nobody is harmed if that counter is a second late.

Saga, not a distributed transaction

When a process spans aggregates in different services, use local transactions plus retries or compensating actions. Our workspace deletion is a small orchestrated saga. In one local transaction identity records the audit entry, inserts one `workspace_purges` row per service and deletes its own data. A `@Scheduled` task then calls `POST /internal/workspace-purge` on each service until it answers, every few minutes for a day and then hourly. The endpoint is idempotent because deleting again deletes nothing, and `WorkspacePurger` plus an optional `WorkspacePurgeHook` handle each service's own rows and files. There is no two-phase commit: each participant owns its data, and a service that is down just gets asked again.

14.8 Anti-corruption layer

An ACL is a translation layer that stops a foreign model leaking into yours. Paddle's vocabulary (transactions, price ids, custom data) and monday's (accounts, app subscriptions) must stop at the edge of billing, while the rest of the system only ever asks "what plan is this workspace on?"

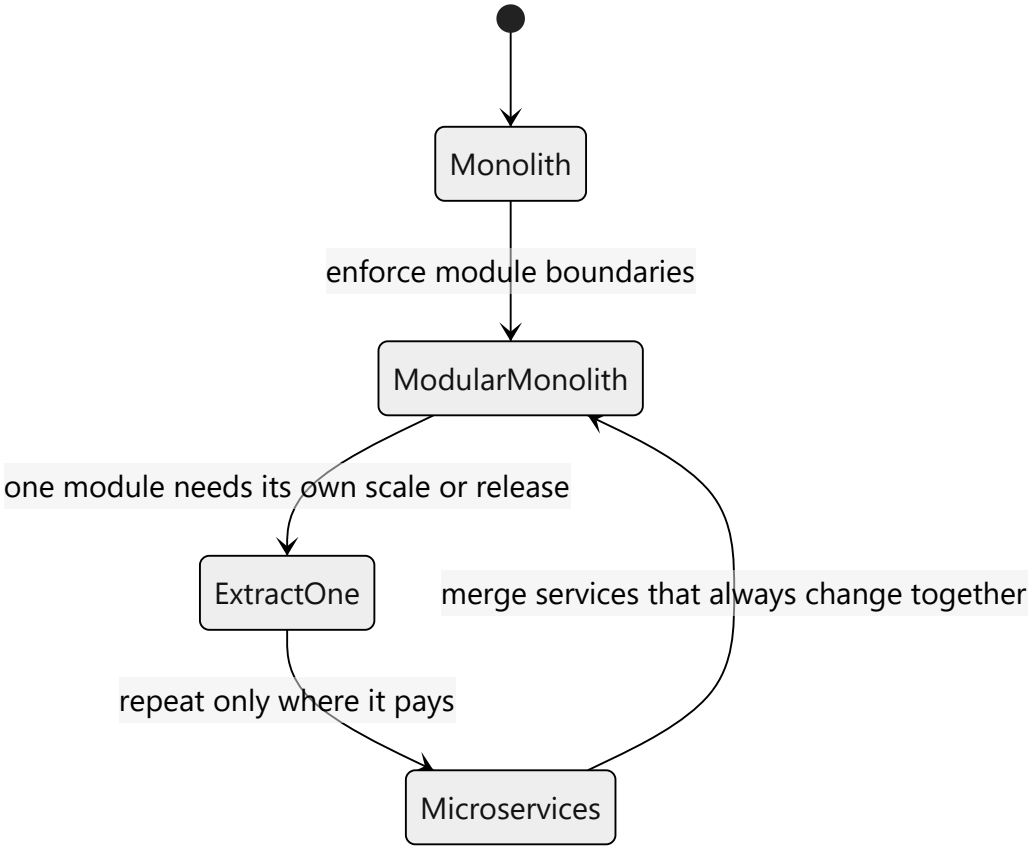


> **RefinePoint:** this is one of our best real boundaries. `PaddleWebhookProcessor` is documented as "Never imports PaddleService — dependency only flows inward". It trusts nothing a buyer's browser could set: the plan comes from the price paid (tagged in Paddle's catalog) and the workspace from the order billing created, never from custom data. Each event is processed once by id, and any failure answers 500 so Paddle resends. monday plugs in as a second source: `MarketplaceSubscriptions.apply(Change)` takes our own record (`orgId` , `marketplace` , `tier` , `active` , `periodEnd`) and returns `APPLIED` , `ENDED` , `KEPT_PADDLE` or `UNCHANGED` . No other service in the system knows Paddle or monday exist. Adding the second provider did not touch a single service outside billing.

ACL building blocks: a **Facade** to simplify the foreign API (`PaddleApi`), an **Adapter** to fit our port, and a **Translator** between the two models. Foreign ids stay only as opaque fields on our entity (`paddleSubscriptionId`).

14.9 Modular monolith vs microservices

	Modular monolith	Microservices
Deployment	One unit	Independent units
Transactions	ACID across modules	Saga, outbox, eventual consistency
Calls	In-process method	Network: latency, retries, partial failure
Refactoring boundaries	IDE refactor	Cross-repo, versioned contracts
Team autonomy	Limited	High
Independent scaling	Limited	Per service
Operations	Simple	Gateway, tracing, broker, many pipelines
Failure isolation	One bad module can take everything down	Bulkheads, if designed



Decide by forces, not fashion: team count, independent release needs, different scaling or runtime needs (a big engine vs a small CRUD app), compliance isolation, and fault isolation. "It is modern" is not a reason. A distributed monolith, where services must deploy together, has the costs of both and the benefits of neither.

Practical rules for a modular monolith: one module per bounded context; each exposes a small `api` package and hides the rest; no JPA relations across modules; one schema per module; modules communicate through interfaces or in-process events. Spring Modulith can verify this and generates documentation from it.

> **RefinePoint:** we began with modules behind ports such as `PlanLimits`; the comment in that interface still mentions the monolith implementing it in the billing module. When we split, those ports became the seams: the interface stayed, and the implementation became a call through `InternalClient`. That is why the split was tractable, and it is a good answer when you are asked how to prepare a monolith for extraction. The services have different runtime profiles too: run-service holds DuckDB and needs big heaps, whereas identity is small. Kafka stayed deliberately narrow: `Topics` says "Kafka is used only where one event has several independent consumers or must survive the consumer being down; everything else between services is a direct API call."

14.10 Technical debt, code review, API design, ArchUnit

Technical debt

Ward Cunningham's metaphor: shipping imperfect code is like taking a loan, and the interest is every future change costing more. Fowler's **debt quadrant** is reckless or prudent crossed with deliberate or inadvertent. Prudent-deliberate ("ship now, we know what it will cost") is legitimate. Reckless-deliberate ("no time for design") is not.

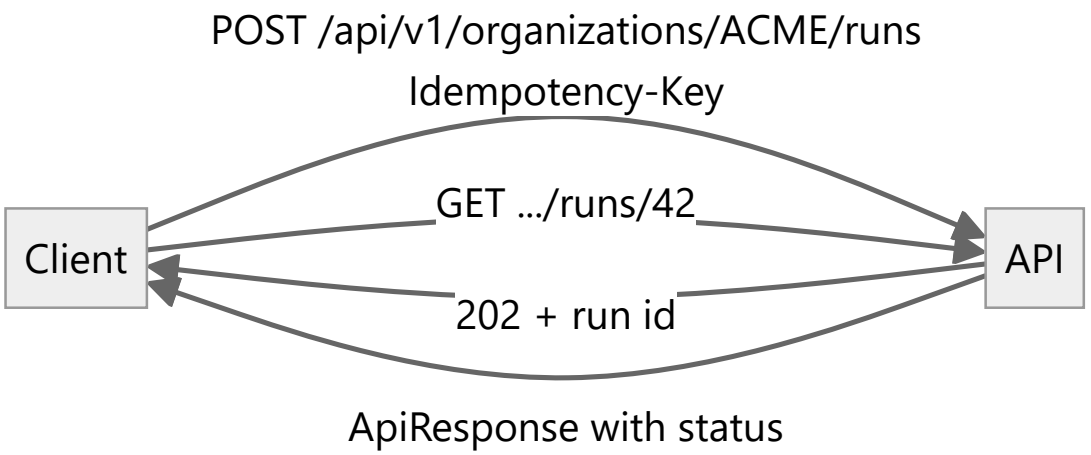
How to manage it: make it visible (tickets with the cost described in business terms such as lead time and incidents); target **hot spots** where churn crosses complexity, because debt in code nobody touches costs nothing; apply the boy-scout rule in the code you are changing; reserve steady capacity rather than a "refactoring sprint" that never gets approved; and write down decisions in ADRs so the reasons survive.

Code review

Reviewer focuses on	Leaves to machines
Correctness, edge cases, failure paths	Formatting, import order
Concurrency, transactions, idempotency	Simple lint rules
Security: authz, tenant isolation, injection	Coverage numbers
Design and naming	Dependency vulnerabilities
Test quality: would it fail if the code were wrong?	Build and test results

Keep pull requests small, because review quality drops sharply with size. Separate blocking from non-blocking comments (`nit:`). Critique the code, not the author. Explain why and offer an alternative. For a senior, the review is also how the team's design knowledge spreads.

API design principles



- Name **resources** with nouns and let HTTP verbs carry the action; scope by tenant in the path.
- **Status codes** carry meaning: 201 created, 202 accepted for async work, 400 validation, 401 vs 403, 404, 409 conflict, 422, 429 with `Retry-After`.
- **Consistent error format** and envelope across services (our `ApiResponse`, RFC 9457 Problem Details is the standard alternative).
- **Idempotency:** PUT and DELETE are idempotent by definition; give POST an `Idempotency-Key` where retries matter.
- **Pagination:** offset for small sets, keyset or cursor for large or changing ones. Always cap page size.
- **Versioning:** `/api/v1`, adding fields is non-breaking; removing or renaming needs a new version and a sunset date.
- **Long-running work** returns 202 plus a resource to poll, instead of holding the connection.
- **Never expose entities;** the DTO is the contract.

ArchUnit: architecture as tests

Rules written down in a wiki decay. ArchUnit expresses them as JUnit tests that read bytecode and fail the build.

```
@AnalyzeClasses(packages = "com.ryfnflow.crm.billing")
class ArchitectureRules {
    @ArchTest static final ArchRule domain_is_framework_free =
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage("org.springframework..",
"jakarta.persistence..");

    @ArchTest static final ArchRule no_cycles =
        slices().matching("com.ryfnflow.crm.billing.(*)..").should().beFreeOfCycles();

    @ArchTest static final ArchRule layers =
layeredArchitecture().consideringOnlyDependenciesInLayers()
        .layer("Web").definedBy("..controller..")
        .layer("Service").definedBy("..service..")
        .layer("Repo").definedBy("..repository..")
        .whereLayer("Web").mayNotBeAccessedByAnyLayer()
        .whereLayer("Repo").mayOnlyBeAccessedByLayers("Service");
```

```
    @ArchTest static final ArchRule no_field_injection =
        noFields().should().beAnnotatedWith(Autowired.class);
}
```

For a legacy codebase, use **FreezingArchRule**, which records current violations and fails only on new ones.

> **Interview tip:** ArchUnit is not in RefinePoint today, and it is better to say so plainly. Our boundaries are enforced by separate repos, so a service physically cannot import another's internals. The rule we would add first is the tenant one: every repository query on a `BaseOrgEntity` must take `orgId`, because that is the bug that would hurt most. Saying what you would add, and why, reads as far more senior than claiming it everywhere.

14.11 Interview questions and answers

Q1. What does "clean code" mean to you at 12 years of experience?

Code whose next change is cheap and safe. That means intent-revealing names, small public surfaces, clear boundaries, and tests that protect behaviour. It is not line counts or a rule that every method be five lines. I judge it by how quickly a new reader can find the place to change and how confident they are afterwards.

Q2. How long should a method be?

As long as one level of abstraction allows. The real test is whether you can name it without "and". A 50-line linear method that reads top to bottom can be better than ten helpers you must jump through, whereas 20 lines mixing SQL, business rules and formatting is too long.

Q3. When are comments good and when are they a smell?

Good: why this decision, a non-obvious constraint, a warning, a public API contract, a link to the incident that caused it. Bad: restating the code, commented-out code, changelogs in the source. A comment that explains *what* usually means the name or structure can improve.

Q4. Checked or unchecked exceptions in application code?

Mostly unchecked in Spring applications. Checked exceptions pollute signatures, do not compose with lambdas, and by default do not roll back `@Transactional`. Use domain-specific runtime exceptions, map them in one `@RestControllerAdvice`, and keep checked exceptions only where a caller can really recover at the very next level.

Q5. Should you return null, Optional, or throw?

Throw when absence is a failure (`findRequired`). Return `Optional` for a legitimately missing single value from a method. Return an empty collection, never null, for lists. Do not use `Optional` for fields or parameters: it is designed as a return type and it complicates serialization and call sites.

Q6. What are the most damaging code smells in your experience?

Shotgun surgery, because it means knowledge is scattered. Divergent change, because unrelated work collides in one class. Temporal coupling, because it fails only in some orders. Primitive obsession, because rules are repeated everywhere. Note that all four are about the cost of change, not appearance.

Q7. What is feature envy, and how do you fix it?

A method that uses another object's data more than its own, so the logic is in the wrong place. Move Method to the data's owner. `run.isStale(now, limit)` beats a service reading `status` and `heartbeatAt` and computing it. Exception: when the logic crosses several aggregates, it belongs in a service.

Q8. Explain how you would refactor a 5,000-line service class in production.

First add characterisation tests around the use cases that matter. Cluster methods and fields to discover hidden classes. Extract one cohesive class at a time, leaving the old class delegating, so the public API does not move. Ship each step separately behind normal releases. Change behaviour only in separate commits. Finally move callers across and delete the delegates. Never a big-bang rewrite: you lose the bug fixes hidden inside the old code.

Q9. What is the strangler fig pattern?

Incrementally replacing a legacy system by routing traffic for one capability at a time to new code behind a facade or gateway, until the old system can be removed. At all times the system works and each step can be rolled back. Branch by abstraction is the same idea inside one codebase.

Q10. How is DRY often misapplied?

By merging code that looks alike but changes for different reasons. Two identical validations belonging to different business rules should stay separate, because the wrong abstraction is more expensive than duplication. DRY concerns duplicated *knowledge*, not duplicated text. The rule of three is a good guard.

Q11. Compare layered and hexagonal architecture.

Layered points dependencies down to the database, so the domain usually depends on persistence. Hexagonal points everything inward to the domain, and technology plugs in through ports owned by the core. Layered is simpler and suits CRUD. Hexagonal is worth its overhead when domain logic is rich, integrations change, or you want to test rules without Spring or a database.

Q12. Inbound port vs outbound port?

An inbound port is what the application offers: a use-case interface called by a controller or listener. An outbound port is what the application needs: an interface it declares and infrastructure implements (`LlmClient`, `Mailer`). Inbound ports are optional in small apps, where a service class can serve; outbound ports give most of the value, since they invert the dependency on technology.

Q13. Is hexagonal the same as clean or onion architecture?

Essentially yes: all enforce inward dependencies with the domain at the centre. Clean names the rings (entities, use cases, interface adapters, frameworks), Onion emphasises domain services rings, Hexagonal emphasises symmetrical ports and adapters on all sides. In an interview, name the shared rule rather than arguing about differences.

Q14. Should the domain model have JPA annotations?

Purists say no: map separate persistence entities. Pragmatists say yes: JPA annotations are metadata, and separate models triple the code for little gain. My default is annotations on the domain entity for ordinary aggregates, and a separate pure model only where the domain is complex or the persistence shape differs greatly from it (for example our rule AST, which is fully framework-free and has no table at all).

Q15. How do you decide whether to put a port in front of something?

When the technology or vendor may change, when you need a test double for something slow or external, or when the owning module should be hidden. A port for a repository that will only ever be Postgres, with a single implementation and no tests benefiting, is ceremony. `LlmClient` earned its port; a CRUD lookup repository did not.

Q16. Package by feature vs package by layer?

By feature groups everything for a capability together, enabling package-private visibility and easy extraction. By layer groups by technical role, so one feature change touches several packages and everything must be public. I choose by feature, with sub-layers inside a large feature if needed. Our `runqueue`, `exports` and `breaks` packages in run-service show it.

Q17. What is a bounded context?

An explicit boundary inside which one model and one language are consistent. The same word can mean different things in different contexts, and that is allowed. "Reconciliation" is a definition in reconciliation-service, an execution in run-service, and a counted unit of usage in billing. Each has its own model instead of one giant shared class.

Q18. Bounded context vs subdomain?

A subdomain is in the **problem** space: what the business does (core, supporting, generic). A bounded context is in the **solution** space: a boundary we choose for a model. Ideally they align one-to-one, but legacy systems often have several subdomains in one context.

Q19. Bounded context vs microservice?

A context is a logical and linguistic boundary; a microservice is a deployment boundary. One context may be several services for scaling reasons, or one service may host several small contexts. A microservice larger than a context usually has model confusion. A microservice much smaller often becomes a chatty distributed monolith.

Q20. How do you find bounded contexts in an existing system?

Listen for language changes: when "customer" means different things to sales and finance, that is a boundary. Use event storming: map events along a timeline and look for pivotal events. Check data ownership (who writes what), change patterns in git (what always changes together), and team structure, thanks to Conway's law.

Q21. Map your project's services to bounded contexts.

"Identity and access is Keycloak plus identity-service. Reconciliation definition and close is reconciliation-service. Run execution, including queue, results, breaks and exports, is run-service, the heart of the core domain. Ingestion and data sources are supporting. Billing is a generic subdomain bought from Paddle and hidden behind an anti-corruption layer. Audit, notifications and AI each own their model. The gateway owns no domain model at all."

Q22. Explain the Customer–Supplier relationship with an example.

Upstream delivers what the downstream needs, and the downstream's requirements influence upstream priorities. Run-service is the customer of reconciliation-service: it reads a `ProcessSnapshot` once at the start of each run. The contract is a published DTO, so reconciliation can evolve its storage (JSONB columns on `recon_processes`) without touching the run engine.

Q23. What is Shared Kernel, and what is its risk?

A small model shared by two or more contexts that must coordinate every change. The risk is hidden coupling and lockstep upgrades: edit `ryfn-platform` and every service must be rebuilt. Keep it small and technical (security, envelope, outbox, base entities), and be suspicious whenever business rules want to move in.

Q24. What is an anti-corruption layer?

A translation layer that keeps an external model from leaking into yours. It usually combines a facade over the foreign API, adapters to your ports, and translators between models. Our `PaddleWebhookProcessor` and `MarketplaceSubscriptions.Change` turn Paddle and monday payloads into our `OrgSubscription`, so no other service knows either vendor exists.

Q25. When is ACL overkill?

When the foreign model fits yours, the integration is small and stable, or you are genuinely conformist by choice. Taking Keycloak's JWT claims and mapping them with one `KeycloakJwtConverter` is enough; a full translation layer around OIDC would add nothing.

Q26. Entity vs value object?

An entity has identity that persists through change: two runs with identical fields are still two runs. A value object is defined only by its attributes, immutable, and replaced rather than modified: two `Money(10, USD)` are the same. Rule of thumb: if you would never ask "which one?", it is a value object.

Q27. How do you persist value objects in JPA?

`@Embeddable` with `@Embedded`, allowing column overrides, for multi-column values. An `AttributeConverter` for a value that fits one column. A JSONB column for a collection of values with no independent identity. Hibernate 6 supports records as embeddables, which removes the need for mutable no-arg classes.

Q28. What is an aggregate and its root?

A cluster of objects treated as one unit for data changes, with one root as the only entry point. Outsiders hold only the root's id; all rules are enforced through its methods; a transaction normally changes one aggregate. An aggregate is defined by consistency needs, not by the object relationships in your UI.

Q29. How do you size an aggregate?

Start with the root alone and add only what must be consistent with it at commit. Ask whether a rule violated for two seconds would hurt anyone. If not, make it eventual. Signs of an oversized aggregate: slow loads, many `OptimisticLockException`s, or two users regularly conflicting on unrelated edits.

Q30. Why reference other aggregates by id?

It prevents accidentally changing two aggregates in one transaction, it stops lazy loading across boundaries, and it allows those aggregates to move to other services. It also keeps memory small. Our `ReconRun` holds a `process_id`, not a `ReconProcess` reference, and it couldn't anyway since it lives in another service's database.

Q31. One transaction per aggregate: is it absolute?

No, it is a strong guideline. In a monolith with low contention, updating two aggregates in one local transaction can be fine, and simpler than eventual consistency. Reasons to break it should be conscious. Across services it becomes absolute, because you only have local transactions.

Q32. What are domain events? Domain event vs integration event?

A domain event records something meaningful that happened, in past tense and immutable. A domain event stays within a bounded context and may carry rich types, usually via `ApplicationEventPublisher`. An integration event crosses contexts, so it is a versioned, serialised public contract, e.g., a Kafka message. Ours: `AlertRaisedEvent` is in-process; `rp.run.completed` is an integration event.

Q33. How do you publish events reliably with aggregates?

Never publish directly to a broker inside the transaction, since the commit and the send can diverge. Use the transactional outbox: write the event in the same database transaction, and have a relay publish it. That is `outbox_events` and `OutboxRelay` in `ryfn-platform`, with idempotent consumers (`ConsumedEvents.firstTime`) on the other side.

Q34. Repository in DDD vs Spring Data repository?

A DDD repository offers collection-like access to an aggregate root, with its interface in the domain. A Spring Data repository generates implementations and exposes far more (`findAll` , `deleteAll` , child entities). If you want purity, define a narrow domain interface, or extend `Repository<T, ID>` and declare only what you need.

Q35. Domain service vs application service?

A domain service holds a business rule that belongs to no single entity and has no infrastructure concerns: matching two records with tolerance. An application service orchestrates a use case: open transaction, load aggregates, call domain methods, save, publish. If an application service has `if` statements about business rules, those are misplaced.

Q36. When is a factory better than a constructor in DDD?

When creating the aggregate requires logic or other inputs, when the constructor would be unreadable, or when the result depends on a decision (which subtype). A static factory method on the root (`Invoice.draftFor(order)`) is the lightest option. A separate factory class fits when it needs dependencies.

Q37. Is an anaemic domain model always bad?

No. For simple CRUD, or for rows that many instances update concurrently with guarded SQL, an anaemic entity is honest and fine. It becomes a problem when complex rules spread across services and duplicate. Our run queue is plain data with SQL guards because a database claim is the real invariant, whereas our rule language is rich.

Q38. How did you use DDD in your project, honestly?

"Strategically, fully: services follow bounded contexts, with ACLs for Paddle and monday, snapshots between definition and run, and events across contexts. Tactically, selectively: the rule language and engine are rich, framework-free models. Much of the supporting CRUD is ordinary entity, service and repository, because the rules there do not justify more."

Q39. What is event storming?

A workshop where domain experts and developers put domain events on a wall in time order, then add commands, actors, policies and aggregates. Pivotal events show context boundaries. It is the fastest way to expose misunderstandings before they harden into code.

Q40. Modular monolith or microservices for a new product?

Modular monolith, unless a concrete force says otherwise: independent team releases, very different runtime profiles, or compliance isolation. It gives fast refactoring and ACID. The prerequisite is enforced boundaries: otherwise you get a big ball of mud that cannot be split later.

Q41. When did you split, and what made it possible?

"The modules were behind ports like `PlanLimits`, so when we split, the interface stayed and its implementation became an `InternalClient` call. Different runtime profiles justified it: run-service needs big heaps for DuckDB, whereas identity is small. Fine-grained data ownership was the hard part: JPA relations across modules had to become ids."

Q42. What is a distributed monolith, and how do you detect one?

Services that are separately deployed but cannot change or run independently. Signs: coordinated releases, shared database tables, long synchronous call chains, one service's outage cascading, and a shared library with business logic. Fix by moving boundaries to real contexts, giving each service its data, and using events where consistency allows.

Q43. How do you keep services decoupled without using events for everything?

Use synchronous calls for queries needing a current answer, with timeouts, and cache short-lived answers. Use events only when a change has several independent consumers or must survive a consumer outage. Our `Topics` class states this: only audit events, run completed and notifications go through Kafka; everything else is a direct API call. Fewer topics means fewer eventual-consistency puzzles.

Q44. How do you handle a process that spans several services?

A saga, orchestrated or choreographed, with each step a local transaction and failures handled by retries or compensation. Our workspace deletion is orchestrated: identity writes one `workspace_purges` row per service in the same transaction as its own deletion, then repeatedly calls `POST /internal/workspace-purge` on each until it answers. The endpoint is idempotent because deleting again deletes nothing.

Q45. Orchestration vs choreography?

Orchestration has a coordinator that knows the steps, making flows visible and easy to change but concentrating knowledge in it. Choreography has services reacting to events, with loose coupling but no single place that shows the flow. Choose orchestration for a business process with a strict sequence, and choreography for independent reactions.

Q46. What is Conway's law, and how does it apply?

Systems mirror the communication structures of the organisations that build them. Use it deliberately: the "inverse Conway maneuver" shapes teams around desired boundaries. As a solo builder I can feel the reverse too: 13 services carry more operational weight than one person would choose purely for team reasons, so I chose them for runtime and isolation reasons.

Q47. How do you manage technical debt with product managers?

Translate it into business terms: "each billing change takes four days instead of one, and this caused two incidents". Tie repayment to feature work in the same area. Reserve a steady share of capacity. Track hot spots

where churn meets complexity. Do not ask for a "refactoring sprint", as it is hard to fund and rarely gets repeated.

Q48. Is all technical debt bad?

No. Prudent, deliberate debt with a known repayment plan, such as validating a market before scaling, is a legitimate business tool. Debt in code that never changes has no interest, so leave it. The dangerous kind is reckless or inadvertent debt that nobody has recorded.

Q49. How do you run a code review for a large PR?

Ask for it to be split first, ideally along refactor and behaviour lines. If it can't be, review the design first (a walkthrough with the author), then the risky parts: data migrations, transactions, concurrency, security. Don't spend comments on formatting. Separate must-fix from suggestions.

Q50. What do you look for first in a review on a multi-tenant system?

Tenant isolation first: does each query scope by `org_id`, and can a user reach another tenant's row by guessing an id? Then authorization, then idempotency and failure paths, then transaction boundaries, and only then style and naming.

Q51. How do you give hard feedback in reviews?

Ask questions rather than give verdicts, describe the risk concretely, and offer an alternative. "If two instances run this, could both claim the row?" lands better than "this is wrong". Prefer a conversation for anything design-level. Praise good solutions as well, since reviews teach both ways.

Q52. REST API design: what makes an API good?

Predictable resources and verbs, correct status codes, one error format, idempotency where retries happen, pagination everywhere, backward-compatible evolution, and no leaking of internal models. Above all, design it from the consumer's use cases, not from the database tables.

Q53. How do you version an API and evolve it safely?

Path version for major breaks (`/api/v1`), additive changes without a version bump, and tolerant readers on clients. Expand and contract: add the new field, migrate consumers, remove the old one. Deprecate with a sunset date and watch usage metrics before removing anything.

Q54. How do you design a long-running operation in REST?

Return `202 Accepted` with a resource id, let clients poll that resource, optionally push on completion. Starting a run in RefinePoint saves a `QUEUED` row and returns right away; workers pick it up, and the UI watches status. This survives restarts and doesn't hold threads or connections.

Q55. Offset vs cursor pagination?

Offset is simple, supports jumping to a page, but gets slower at depth and shifts when rows change. Keyset/cursor is stable and fast on an indexed column, but it allows only next/previous. Use keyset for feeds and large result tables, and offset for small admin lists.

Q56. What is ArchUnit, and what rules are worth enforcing?

A library that analyses bytecode so architecture rules can run as JUnit tests. Worth enforcing: domain free of frameworks, no package cycles, layer access directions, no field injection, naming conventions, no cross-feature access to internals. For a legacy code base use `FreezingArchRule` to block only new violations.

Q57. ArchUnit vs Java modules vs separate Maven modules?

Maven modules give hard compile-time dependency direction. JPMS gives runtime-enforced encapsulation, but it is painful with reflection-heavy Spring. ArchUnit gives flexible and precise rules, but is checked only at test time. A common blend is Maven modules for large boundaries, and ArchUnit or Spring Modulith for rules inside.

Q58. What is an ADR, and why write one?

An Architecture Decision Record is a short note: context, decision, alternatives, consequences. It keeps the *why* alive. Six months later the choice to use `SKIP LOCKED` instead of a broker for the run queue doesn't get reversed by someone who has forgotten the reasons.

Q59. Give an example where you chose not to apply a pattern from this chapter.

"We have no hexagonal layering around plain CRUD such as lookup tables: entity, repository, controller and a little service are enough, and a port would have one implementation forever. We also keep `ReconProcess` mappings as JSONB rather than as many child entity tables, because the definition is always read and written whole. That simplifies the aggregate and avoids mapping layers."

Q60. Design question: you inherit a big ball of mud. What are your first 90 days?

Map it first: hot spots from git, runtime dependencies, incident history. Add tests around revenue paths and fix the build pipeline so changes are safe. Draw target contexts with domain experts. Establish seams with facades, and enforce new rules using frozen ArchUnit tests. Extract the highest-value boundary, deliver one visible win, then repeat. No rewrite announcement.